

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

The rapid shift toward digitalization in education, governance, and organizational systems has significantly increased the reliance on electronic certificates and digital credentials. However, this transformation has also introduced critical security challenges such as certificate forgery, identity theft, and unauthorized document manipulation. Traditional certificate verification systems typically depend on centralized databases and manual validation processes, which are highly vulnerable to tampering, single-point failures, and insider attacks. As a result, there is a growing need for advanced, intelligent, and secure frameworks that can ensure authenticity, integrity, and trust in digital certificate management systems.

With the advancement of technologies such as artificial intelligence, cryptography, and distributed ledger systems, new opportunities have emerged to strengthen digital security infrastructure. Biometric authentication techniques, particularly facial recognition, have gained importance in identity verification due to their non-intrusive and reliable nature. At the same time, cryptographic methods such as AES (symmetric encryption) and ECC (asymmetric encryption) provide robust mechanisms for securing sensitive data during storage and transmission. However, when used independently, these technologies still face limitations in scalability, transparency, and tamper resistance, especially in large-scale certificate management systems. To address these limitations, blockchain technology has emerged as a powerful solution for creating decentralized and immutable record-keeping systems. However, when used independently, cryptographic solutions cannot fully address challenges related to transparency, trust, and decentralized verification, especially in large-scale and multi-institutional environments.

1.2 MOTIVATION

The primary motivation behind this project arises from the increasing incidents of certificate forgery, identity fraud, and unauthorized document manipulation in academic institutions, government agencies, and corporate environments. Traditional verification systems are often centralized, manual, and time-consuming, making them highly vulnerable to tampering, insider threats, and data breaches. These systems rely heavily on human intervention and paper-based validation, which not only increases operational delays but also reduces the reliability and trustworthiness of the verification process. As a result, organizations face significant challenges in ensuring the authenticity of credentials, leading to financial losses, reputational damage, and serious security risks.

As digital certificates become widely used for educational qualifications, employment verification, and identity proofing, ensuring their authenticity and integrity has become a critical challenge. The rapid digital transformation across sectors has significantly increased dependency on electronic records. However, the absence of robust and scalable verification mechanisms has created opportunities for malicious actors to exploit system vulnerabilities. This growing gap between digital adoption and security assurance strongly motivates the need for an intelligent, automated, and secure verification framework that can operate efficiently in real-time environments.

Another key motivation is the advancement of emerging technologies such as artificial intelligence, blockchain, and cryptographic systems, which provide powerful tools to enhance security and trust in digital ecosystems. AI-based facial recognition enables reliable identity verification by ensuring that only authorized individuals can access or validate certificates, thereby minimizing impersonation and unauthorized access. At the same time, cryptographic techniques ensure confidentiality and integrity of sensitive data, protecting it from unauthorized disclosure and modification during storage and transmission.

1.3 PROPOSED SOLUTION

The proposed solution introduces a secure and intelligent certificate verification system that integrates facial biometric authentication, hybrid encryption, and blockchain technology into a unified framework. The system begins with user identity verification using facial recognition based on the Grassmann algorithm, ensuring that only legitimate users can access or manage certificate-related operations. This biometric layer significantly reduces the risk of impersonation and unauthorized access, thereby strengthening the identity assurance mechanism of the system. It replaces traditional password-based or manual verification methods with a more reliable and automated authentication approach.

To ensure data security and confidentiality, the system employs a hybrid encryption technique combining AES (Advanced Encryption Standard) and ECC (Elliptic Curve Cryptography). AES is used for fast and efficient encryption of certificate data, while ECC is utilized for securely encrypting and managing cryptographic keys. This dual-layer encryption approach ensures that sensitive certificate information remains protected during storage, transmission, and retrieval processes. Even if data is intercepted, it remains unreadable without proper decryption keys, thereby enhancing the overall resilience of the system against cyber threats.

Furthermore, the solution leverages blockchain technology to create a decentralized and tamper-proof ledger for storing certificate records. Each certificate is hashed, encrypted, and recorded as a blockchain transaction, ensuring immutability and traceability across the system. This eliminates the risks associated with centralized storage, such as data alteration or unauthorized modifications. Each certificate is converted into a unique cryptographic hash and stored as a transaction in the blockchain network along with essential metadata such as timestamp, certificate ID, and issuer details.

1.4 OBJECTIVES

To Prevent Certificate Forgery and Fraud

The primary objective of this system is to eliminate the generation and usage of fake certificates and prevent any unauthorized modifications to existing records. By integrating advanced verification mechanisms, the system ensures that only genuine and authenticated certificates are accepted by institutions, employers, and other stakeholders.

To Implement Secure User Authentication

Another important objective is to strengthen identity verification by implementing facial biometric authentication based on the Grassmann algorithm. This approach ensures that only authorized users can access, upload, or verify certificate data within the system. By linking user identity with biometric features, the system minimizes the risks of impersonation and unauthorized access, thereby improving overall security.

To Ensure Strong Data Security Using Hybrid Encryption

The system aims to protect sensitive certificate data through a hybrid encryption technique that combines Advanced Encryption Standard (AES) and Elliptic Curve Cryptography (ECC). AES provides fast and efficient data encryption, while ECC ensures secure key exchange and management. This combination guarantees confidentiality, data integrity, and protection against cyber threats during both storage and transmission.

To Build a Tamper-Proof Certificate Storage System

The system focuses on using blockchain technology to store certificate records in a decentralized and immutable ledger. This ensures transparency, traceability, and resistance to data tampering.

To Enable Efficient and Trusted Certificate Verification

The system aims to provide a fast, reliable, and user-friendly verification process using OTP and hash-based validation. This improves trust, reduces manual effort, and ensures secure certificate authentication.

1.5 SCOPE

The scope of this system lies in developing a highly secure, intelligent, and scalable certificate verification framework that can be widely adopted across educational institutions, government organizations, and corporate sectors. With the rapid growth in digital transformation and the increasing reliance on electronic credentials, the system aims to replace traditional manual and centralized verification methods with an advanced, automated, and secure solution. This transition not only reduces processing time but also minimizes human errors and enhances overall reliability.

By integrating advanced technologies such as facial biometric authentication, hybrid encryption, and blockchain, the system ensures that both certificate issuance and verification processes are secure, efficient, and tamper-resistant. This makes it highly suitable for real-world applications such as academic degree verification, employee credential validation, digital identity management systems, and government document authentication. It can also be used by recruitment agencies and multinational organizations to quickly verify candidate credentials without relying on lengthy background checks.

In a broader perspective, the system extends its scope to emerging digital ecosystems where trust, transparency, and security play a crucial role. The incorporation of AI-based facial recognition significantly improves identity verification accuracy by ensuring that certificates are accessed and validated only by authorized individuals. At the same time, hybrid cryptographic techniques combining AES and ECC provide strong data protection, ensuring confidentiality and integrity in both cloud-based and distributed environments. The use of hybrid encryption techniques combining AES and ECC ensures robust protection of sensitive data during both storage and transmission, making the system suitable for both cloud-based and decentralized environments.

CHAPTER 2

LITERATURE SURVEY

2.1 ACADEMIC CERTIFICATE FRAUD DETECTION SYSTEM FRAMEWORK USING BLOCKCHAIN TECHNOLOGY ^[6]

Lutfiani, Ninda et al. [6] proposed a blockchain-based framework designed to detect and prevent academic certificate fraud in modern digital environments. The study focuses on addressing the increasing misuse of fake academic credentials and highlights the limitations of traditional centralized verification systems, which are often vulnerable to data manipulation, security breaches, and lack of transparency.

The proposed framework utilizes a decentralized blockchain network to ensure the authenticity, integrity, and immutability of academic certificates. By storing certificate records in a distributed ledger, the system eliminates dependency on a single authority and significantly reduces the risk of unauthorized modifications. Once the certificate data is recorded on the blockchain, it becomes permanent and tamper-proof, ensuring a high level of security and reliability.

In this approach, each certificate is assigned a unique digital identity (hash value), which acts as a fingerprint for verification across the network. Once the certificate data is recorded, it becomes permanent and tamper-proof, providing a high level of security and trust among stakeholders.

Advantages:

- Ensures high security and immutability using blockchain technology.
- Provides real-time verification of academic certificates.
- Reduces dependency on centralized authorities.

Disadvantages:

- Implementation complexity and high initial setup cost.
- Requires technical expertise and infrastructure for deployment.

2.2 FAKE CERTIFICATE DETECTION BY USING BLOCKCHAIN ^[4]

Chanekar, Niraj et al. [4] proposed a blockchain-based approach to detect fake certificates and ensure document authenticity in both academic and professional domains. The study addresses the increasing problem of forged certificates and emphasizes the weaknesses of traditional verification systems, which are often manual, time-consuming, and vulnerable to data manipulation and fraud.

The proposed framework leverages blockchain technology to create a secure, decentralized, and tamper-proof environment for certificate validation. Instead of relying on centralized databases, the system stores certificate-related information in a distributed ledger, ensuring that data cannot be altered once it is recorded. This significantly enhances the integrity and reliability of certificate records.

In this approach, certificate metadata is stored on the blockchain, and each certificate is assigned a unique cryptographic hash value, which acts as a digital fingerprint. This enables accurate and efficient verification, as any modification in the certificate data will result in a change in the hash value, making tampering easily detectable. As a result, any attempt to tamper with the certificate can be easily detected during the verification process. This enables accurate, fast, and reliable validation without the need for manual intervention or third-party verification.

Advantages:

- Secure and tamper-proof certificate verification using blockchain
- Enables real-time validation without third-party involvement.
- Reduces manual verification effort and improves transparency.

Disadvantages:

- Requires blockchain infrastructure and technical expertise.
- Scalability and performance issues in large-scale implementations.

2.3 BLOCKCHAIN-BASED STUDENT CERTIFICATE MANAGEMENT AND SYSTEM SHARING USING HYPERLEDGER FABRIC PLATFORM ^[5]

Ghani, Rana F. et al. [5] proposed a blockchain-based certificate management system utilizing the Hyperledger Fabric platform to ensure secure sharing and verification of academic credentials. The study focuses on enhancing the efficiency, reliability, and security of certificate issuance and validation processes by adopting a permissioned blockchain model, which provides better control and privacy compared to public blockchains.

The proposed framework is built on Hyperledger Fabric, a permissioned blockchain platform that allows only authorized participants to join the network. In this system, educational institutions act as trusted nodes responsible for issuing and managing certificates.

This controlled access mechanism ensures that only verified entities can participate in certificate-related transactions, thereby improving the overall trustworthiness of the system. Certificates are stored as secure blockchain transactions, protected using advanced cryptographic techniques such as encryption and hashing. This controlled access mechanism enhances trust and reduces the risk of fraudulent activities. The system also supports role-based access control, allowing different participants to have specific permissions based on their roles within the network.

Advantages:

- Ensures secure and private certificate management using permissioned blockchain.
- Supports automated validation through smart contracts.
- Enhances scalability and data consistency across institutions.

Disadvantages:

- Requires complex setup and maintenance of blockchain network.
- Limited flexibility compared to public blockchain systems

2.4 DYNAMIC SECURE ACCESS CONTROL AND DATA SHARING THROUGH TRUSTED DELEGATION AND REVOCATION IN A BLOCKCHAIN-ENABLED CLOUD-IoT ENVIRONMENT ^[1]

Alshehri et al. [1] proposed a blockchain-based secure access control mechanism designed for cloud and IoT environments. The study focuses on enabling dynamic delegation and revocation of access rights, ensuring secure and efficient data sharing across distributed systems. It addresses the limitations of traditional access control systems, which often rely on centralized authorities and are vulnerable to security breaches, lack of transparency, and inefficient management of permissions.

The proposed framework leverages blockchain technology to maintain an immutable and decentralized record of access control policies. By storing access rules and transactions on the blockchain, the system ensures that all operations are transparent, traceable, and resistant to unauthorized modifications. This significantly enhances trust among users and eliminates dependency on a single centralized authority.

One of the key features of the system is its ability to support secure data sharing across cloud-IoT networks. Users can share sensitive data with other entities in a controlled and secure manner using a decentralized architecture. Cryptographic techniques are used to protect data during transmission and storage, ensuring that only authorized users can access the information.

Advantages:

- Provides dynamic and fine-grained access control in distributed systems.
- Ensures secure data sharing with real-time access revocation.
- Enhances transparency and accountability using blockchain logs.

Disadvantages:

- High computational overhead due to blockchain operations.
- Complexity in integrating with existing cloud and IoT systems.

2.5 AN EFFICIENT AND SECURE ENERGY TRADING APPROACH WITH MACHINE LEARNING TECHNIQUE AND CONSORTIUM BLOCKCHAIN ^[3]

Ashfaq et al. [3], proposed a secure energy trading system using machine learning and consortium blockchain technology. The study addresses issues of trust, security, and transparency in energy trading systems by introducing a decentralized framework for transaction validation.

The system uses blockchain to record and validate energy transactions securely, enabling peer-to-peer energy exchange without centralized control. Machine learning techniques are applied to optimize trading decisions and detect anomalies in transactions. Smart contracts automate transaction execution and settlement processes, ensuring efficiency and reliability.

The consortium blockchain model restricts participation to authorized entities, improving control and scalability. Additionally, cryptographic validation prevents fraudulent transactions and ensures transparency in pricing and energy distribution. The framework supports real-time processing, enhances data integrity, and integrates AI for intelligent decision-making, making it a robust solution for modern energy trading ecosystems. The framework also utilizes smart contracts to automate transaction execution and settlement processes. These smart contracts operate based on predefined conditions, ensuring that transactions are executed only when the specified criteria are met.

Advantages:

- Secure and transparent peer-to-peer energy trading system.
- Integration of machine learning improves decision-making.
- Scalable and efficient due to consortium blockchain model.

Disadvantages:

- Requires complex integration of AI and blockchain technologies.
- High implementation and maintenance cost.

2.6 RETHINKING CERTIFICATION FOR TRUSTWORTHY MACHINE-LEARNING-BASED APPLICATIONS ^[2]

Anisetti, Marco et al. [2], proposed a certification framework aimed at improving the trustworthiness and reliability of machine-learning-based applications. The study highlights challenges in validating AI systems, especially in safety-critical domains. The framework introduces structured certification mechanisms to evaluate performance, robustness, and security of machine learning models. It emphasizes standardized validation processes to ensure reliability and discusses risks such as model bias, adversarial attacks, and lack of transparency. It ensures compliance with predefined quality standards and promotes transparency in AI decision-making. Additionally, it supports continuous monitoring of deployed models and ensures reproducibility of results, thereby enhancing trust and accountability in AI systems.

Another important aspect of the study is its emphasis on transparency and explainability in AI decision-making. The framework encourages the use of interpretable models and documentation techniques that allow stakeholders to understand how predictions are generated. The system supports systematic evaluation under various operational conditions by incorporating comprehensive testing methodologies. These include stress testing, scenario-based validation, and performance benchmarking to assess how models behave under different inputs and environments.

Advantages:

- Improves reliability and trust in machine learning systems.
- Provides standardized evaluation and certification processes.
- Supports continuous monitoring and validation.

Disadvantages:

- Difficult to standardize across diverse AI applications.
- Requires significant effort for implementation and compliance.

2.7 CERTIFICATION SYSTEMS FOR MACHINE LEARNING: LESSONS FROM SUSTAINABILITY ^[7]

Matus, Kira J. M., and Michael Veale [7] explored certification systems for machine learning by drawing valuable insights from sustainability certification models. The study focuses on improving accountability, transparency, and trust in artificial intelligence (AI) systems by introducing structured certification frameworks similar to those used in environmental and sustainability domains.

The research emphasizes the importance of establishing standardized evaluation criteria for machine learning models to ensure consistency, reliability, and fairness. It highlights that, just like sustainability certifications (such as eco-labeling), AI systems also require clear benchmarks and guidelines to evaluate their performance, ethical compliance, and societal impact. This approach helps organizations ensure that their AI systems meet defined quality and ethical standards before deployment.

Furthermore, the study compares AI certification processes with environmental sustainability frameworks and identifies key challenges in developing universally accepted certification standards. These challenges include the dynamic nature of AI systems, lack of transparency, and differences in regulatory policies across regions. This makes it difficult to maintain consistent certification over time. Additionally, the lack of transparency in many AI algorithms, creates barriers to effective evaluation and auditing.

Advantages:

- Promotes ethical and transparent AI development.
- Supports standardized certification and auditing processes.
- Enhances trust among stakeholders.

Disadvantages:

- Challenges in global standardization of certification frameworks.
- Implementation may vary across different sectors.

2.8 A SYSTEMATIC LITERATURE REVIEW ON BLOCKCHAIN-BASED SYSTEMS FOR ACADEMIC CERTIFICATE VERIFICATION [8]

Rustemi et al. [8] presented a comprehensive and systematic review of blockchain-based systems designed for academic certificate verification. The study provides an in-depth analysis of various existing approaches and evaluates their effectiveness in addressing the growing problem of certificate forgery and fraudulent credential usage in educational institutions.

The research highlights how blockchain technology ensures the immutability, transparency, and security of academic records by storing certificate data in a decentralized ledger. Since blockchain records cannot be altered once added, it significantly reduces the risk of data tampering and unauthorized modifications. The study also explains how decentralized storage eliminates dependency on a single authority, thereby improving trust and reliability among stakeholders.

Furthermore, the paper compares different system architectures adopted in blockchain-based certificate verification systems, including public, private, and consortium blockchains. It analyzes their advantages and limitations in terms of scalability, cost, performance, and access control. It highlights that while blockchain offers strong security benefits, challenges such as high computational overhead, storage limitations, and network latency must be addressed for large-scale implementation.

Advantages:

- Provides comprehensive analysis of existing blockchain systems.
- Highlights strengths and limitations for future research.
- Supports adoption of secure verification technologies.

Disadvantages:

- Does not propose a new system (review-based study).
- Limited to analysis of existing implementations.

2.9 DIGITAL CERTIFICATE SECURITY: A BLOCKCHAIN-BASED APPROACH FOR FRAUD PREVENTION AND VERIFICATION ^[10]

Zaman, Nihat et al. [10] proposed a blockchain-based approach aimed at enhancing digital certificate security and preventing fraud in academic and professional environments. The study focuses on addressing the vulnerabilities present in traditional certificate verification systems, which are often centralized, prone to data manipulation, and lack transparency. The proposed system utilizes blockchain technology to store certificate data in a decentralized and tamper-proof ledger. Each certificate is assigned a unique cryptographic hash value, which acts as a digital fingerprint for verification. This ensures that any alteration in the certificate data can be easily detected, thereby maintaining data integrity and authenticity.

By leveraging the decentralized nature of blockchain, the system eliminates the risks associated with centralized storage, such as single-point failures, unauthorized access, and data breaches. All certificate transactions are recorded in an immutable ledger, ensuring transparency and traceability of records. Furthermore, the framework integrates smart contracts to automate the certificate issuance and verification processes. These smart contracts execute predefined conditions, ensuring that only valid and authorized certificates are accepted.

Advantages:

- Strong security through cryptographic hashing and blockchain.
- Real-time and automated certificate verification.
- Reduces fraud and duplication risks.

Disadvantages:

- Requires blockchain infrastructure setup.
- Performance issues may arise with large-scale data.

2.10 A BLOCKCHAIN-BASED CONCEPTUAL MODEL TO ADDRESS EDUCATIONAL CERTIFICATE VERIFICATION CHALLENGES IN TANZANIA ^[9]

Said et al. [9], proposed a blockchain-based conceptual model to address challenges in educational certificate verification systems, particularly in Tanzania. The study identifies issues such as certificate forgery, manual delays, and centralized system limitations. The framework introduces blockchain as a decentralized solution to securely store academic records in an immutable ledger. It enables real-time verification by authorized stakeholders and reduces dependency on manual processes. Cryptographic hashing ensures data integrity and authenticity, while secure data sharing improves collaboration between institutions. The system minimizes administrative workload and prevents unauthorized duplication of certificates.

It also supports scalability for national-level implementation and promotes digital transformation in education systems. Overall, the model enhances efficiency, transparency, and security in certificate verification workflows. This eliminates the need for time-consuming manual verification processes and reduces administrative delays. The decentralized nature of the system also removes dependency on a single authority, thereby minimizing risks such as system failures, data breaches, and unauthorized modifications.

Advantages:

- Reduces fraud and manual verification delays.
- Ensures secure and transparent certificate management.
- Scalable for large educational systems.

Disadvantages:

- Requires infrastructure development for nationwide adoption.
- Initial implementation cost can be high.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

The existing certificate management systems are primarily based on centralized architectures in which institutions store and manage certificates using local databases or cloud-based servers. In such systems, users upload their documents, including academic certificates, identity proofs, and professional licenses, which are then stored, processed, and retrieved through centralized verification portals. These systems aim to simplify record management and improve accessibility compared to traditional paper-based methods. In many cases, certificates are converted into QR codes to facilitate quick verification. Authorized users or organizations can scan these QR codes to access the certificate details and validate authenticity. While this method enhances convenience and reduces manual effort, it still relies heavily on the integrity and availability of centralized databases. If the central server is compromised, corrupted, or unavailable, the entire verification process may be affected.

Authentication in most existing systems is implemented using basic security mechanisms such as username-password login or One-Time Password (OTP) verification. Although these methods provide a certain level of protection, they are not sufficient to prevent advanced security threats. Credentials can be stolen through phishing attacks, brute-force methods, or data breaches, leading to unauthorized access and misuse of sensitive information. Some modern systems attempt to improve security by incorporating blockchain technology. In such cases, only partial data such as QR code hashes or certificate metadata is stored on the blockchain, while the actual documents remain in centralized storage.

3.1.1 DISADVANTAGES OF EXISTING SYSTEM

- Heavily dependent on centralized systems, making it vulnerable to single-point failures and data tampering.
- Lacks advanced authentication mechanisms, increasing the risk of impersonation and unauthorized access.
- Requires continuous internet connectivity, limiting usability in low-network or remote areas.
- Faces privacy and security risks when third-party verifiers access sensitive certificate data.
- Susceptible to device-level attacks such as OTP interception and unauthorized document access.

3.2 PROPOSED SYSTEM

The proposed system introduces a secure, intelligent, and scalable certificate verification framework that integrates advanced technologies such as facial biometric authentication, hybrid encryption, and blockchain. This system is designed to overcome the limitations of traditional certificate verification methods by providing a reliable, automated, and tamper-resistant solution. It ensures authenticity, enhances data security, and builds trust among users, educational institutions, and organizations.

The system begins with a robust identity verification mechanism using facial recognition based on the Grassmann algorithm. This biometric authentication method analyzes facial features in a mathematical subspace, ensuring accurate and consistent identification even under varying environmental conditions. Unlike traditional authentication methods such as usernames and passwords, which are vulnerable to attacks, this approach significantly reduces the risk of impersonation and unauthorized access. By linking system access directly to unique biometric characteristics, it establishes a strong and reliable authentication layer.

To ensure the security of sensitive certificate data, the system employs a hybrid encryption mechanism that combines Advanced Encryption Standard (AES) and Elliptic Curve Cryptography (ECC). AES is used for encrypting large volumes of certificate data efficiently due to its high speed and performance. ECC, on the other hand, is utilized for secure key generation and exchange, offering strong security with smaller key sizes. This dual-layer encryption approach guarantees confidentiality, integrity, and protection against cyber threats. Even if the data is intercepted during transmission or storage, it remains unreadable without the proper decryption keys.

Furthermore, the system leverages blockchain technology to create a decentralized and tamper-proof storage infrastructure for certificate records. Each certificate is transformed into a unique cryptographic hash value, which is stored as a transaction on the blockchain network along with relevant metadata such as timestamp and issuer details. Due to the immutable nature of blockchain, once the data is recorded, it cannot be modified or deleted, ensuring long-term integrity and reliability of records.

3.2 1 ADVANTAGES OF PROPOSED SYSTEM

- Provides strong security using hybrid AES and ECC encryption techniques.
- Ensures accurate identity verification through facial recognition (Grassmann algorithm).
- Offers tamper-proof and transparent storage using blockchain technology.
- Enables fast and reliable certificate verification using OTP and hash-based validation.
- Supports real-time verification, improving speed and operational efficiency.
- Scalable and adaptable for large-scale institutional or national-level deployment.

CHAPTER 4

SYSTEM SPECIFICATIONS

4.1 HARDWARE REQUIREMENTS

- Processor : Quad core processor (Intel i5 / AMD Ryzen 5 or higher)
- RAM : 8 GB (Minimum), 16 GB recommended
- Hard disk : 512 GB SSD (1 TB preferred for blockchain storage)
- Network : High-speed broadband (10 Mbps or higher)
- GPU (Optional) : For cryptographic computation and acceleration
- Server Machine : Dedicated system for blockchain node (optional)
- Backup Storage : External HDD / Cloud backup

4.2 SOFTWARE REQUIREMENTS

- Operating system : Windows 10 / Windows 11 / macOS (64-bit OS)
- Front End : Html, CSS, JAVASCRIPT
- Back End : Python, Flask
- Data base : MySQL SERVER / MongoDB
- IDLE : Python 2.7 IDLE

CHAPTER 5

SOFTWARE DESCRIPTION

5.1 FRONTEND DEVELOPMENT

HTML is the standard markup language used to create the structure and content of web pages. It defines the basic elements such as headings, paragraphs, images, links, tables, and forms that appear in a browser. HTML works by using tags to organize content into a hierarchical structure, allowing developers to build well-formatted and meaningful web documents. It provides the backbone of any website, ensuring that content is displayed properly and can be interpreted by browsers and search engines. Modern HTML, particularly HTML5, introduces semantic elements such as <header>, <footer>, <section>, and <article>, which improve readability, accessibility, and search engine optimization.

In addition to structuring content, HTML integrates seamlessly with CSS and JavaScript to create interactive and visually appealing web applications. It supports multimedia elements like audio and video without requiring external plugins, making web pages more dynamic. Forms in HTML enable user input through text fields, radio buttons, checkboxes, and submit buttons, which are essential for login systems, registrations, and data collection in web applications. Overall, HTML serves as the foundational layer of front-end development, providing a structured framework upon which styling and interactivity are built.

CSS (Cascading Style Sheets)

CSS is a styling language used to control the presentation and layout of HTML elements on a web page. It allows developers to apply colors, fonts, spacing, borders, and positioning to create visually attractive and responsive designs. By separating content (HTML) from design (CSS), developers can maintain consistency across multiple web pages while improving maintainability and scalability. CSS supports different types of styling methods, including inline

styles, internal stylesheets, and external stylesheets, with external stylesheets being the most efficient for large projects.

Modern CSS includes powerful features such as Flexbox and Grid, which enable responsive and flexible layouts for different screen sizes and devices. Media queries allow developers to design websites that automatically adapt to desktops, tablets, and smartphones. Animations, transitions, and hover effects enhance user experience by adding smooth visual interactions. CSS plays a crucial role in improving usability, branding, and overall user interface design, ensuring that web applications are not only functional but also aesthetically appealing.

JavaScript

JavaScript is a scripting language used to add interactivity and dynamic behavior to web pages. Unlike HTML and CSS, which focus on structure and design, JavaScript enables real-time user interaction, data validation, animations, and content updates without reloading the page. It allows developers to manipulate the Document Object Model (DOM), respond to user events such as clicks and key presses, and dynamically modify content. This makes JavaScript essential for creating modern web applications such as dashboards, online forms, games, and interactive platforms.

Furthermore, JavaScript supports asynchronous programming through features like AJAX and Fetch API, enabling communication with servers without refreshing the entire webpage. It is widely used with frameworks and libraries such as React, Angular, and Vue.js to build complex single-page applications. JavaScript also plays a key role in client-side validation, improving performance and reducing server load.

JavaScript also supports client-side data validation, which ensures that user input is checked before being sent to the server. This improves application performance by reducing unnecessary server requests and enhances user experience by providing immediate feedback.

5.2 BACKEND DEVELOPMENT

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace. It provides constructs that enable clear programming on both small and large scales. In July 2018, Van Rossum stepped down as the leader in the language community. Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library. Python interpreters are available for many operating systems. CPython, the reference implementation of Python, is open source software and has a community-based development model, as do nearly all of Python's other implementations.

Python and CPython are managed by the non-profit Python Software Foundation. Rather than having all of its functionality built into its core, Python was designed to be highly extensible. This compact modularity has made it particularly popular as a means of adding programmable interfaces to existing applications. Van Rossum's vision of a small core language with a large standard library and easily extensible interpreter stemmed from his frustrations with ABC, which espoused the opposite approach. While offering choice in coding methodology, the Python philosophy rejects exuberant syntax (such as that of Perl) in favor of a simpler, less-cluttered grammar. As Alex Martelli put it: "To describe something as 'clever' is not considered a compliment in the Python culture." Python's philosophy rejects the Perl "there is more than one way to do it" approach to language design in favour of "there should be one—and preferably only one—obvious way to do it".

Python's developers strive to avoid premature optimization, and reject patches to non-critical parts of CPython that would offer marginal increases in speed at the cost of clarity.[When speed is important, a Python programmer can

move time-critical functions to extension modules written in languages such as C, or use PyPy, a just-in-time compiler. CPython is also available, which translates a Python script into C and makes direct C-level API calls into the Python interpreter.

An important goal of Python's developers is keeping it fun to use. This is reflected in the language's name a tribute to the British comedy group Monty Python and in occasionally playful approaches to tutorials and reference materials, such as examples that refer to spam and eggs (from a famous Monty Python sketch) instead of the standard for and bar. A common neologism in the Python community is *pythonic*, which can have a wide range of meanings related to program style. To say that code is *pythonic* is to say that it uses Python idioms well, that it is natural or shows fluency in the language, that it conforms with Python's minimalist philosophy and emphasis on readability.

In contrast, code that is difficult to understand or reads like a rough transcription from another programming language is called *unpythonic*. Users and admirers of Python, especially those considered knowledgeable or experienced, are often referred to as Pythonists, Pythonistas, and Pythoneers. Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. Its high-level built in data structures, combined with dynamic typing and dynamic binding, make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together.

Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance. Python supports modules and packages, which encourages program modularity and code reuse. The Python interpreter and the extensive standard library are available in source or binary form without charge for all major platforms, and can be freely distributed. Often, programmers fall in love with Python because of the increased productivity it provides. Since there is no compilation step, the edit-test-debug cycle is

incredibly fast. Debugging Python programs is easy: a bug or bad input will never cause a segmentation fault.

Instead, when the interpreter discovers an error, it raises an exception. When the program doesn't catch the exception, the interpreter prints a stack trace. A source level debugger allows inspection of local and global variables, evaluation of arbitrary expressions, setting breakpoints, stepping through the code a line at a time, and so on. The debugger is written in Python itself, testifying to Python's introspective power. Python's initial development was spearheaded by Guido van Rossum in the late 1980s. Today, it is developed by the Python Software Foundation. Because Python is a multiparadigm language, Python programmers can accomplish their tasks using different styles of programming: object oriented, imperative, functional or reflective. Python can be used in Web development, numeric programming, game development, serial port access and more.

There are two attributes that make development time in Python faster than in other programming languages:

1. Python is an interpreted language, which precludes the need to compile code before executing a program because Python does the compilation in the background. Because Python is a high-level programming language, it abstracts many sophisticated details from the programming code. Python focuses so much on this abstraction that its code can be understood by most novice programmers.
2. Python code tends to be shorter than comparable codes. Although Python offers fast development times, it lags slightly in terms of execution time. Its simple syntax reduces the amount of code required to perform tasks, thereby increasing development speed and productivity. This readability also makes collaboration and maintenance easier.

5.3 API TOOLS AND TESTING

TensorFlow is an open-source machine learning framework developed by Google Brain Team. It is one of the most popular libraries for building and training machine learning models, especially deep neural networks. TensorFlow allows developers to build complex models with ease, including image and speech recognition, natural language processing, and more. One of the key features of TensorFlow is its ability to handle large-scale datasets and complex computations, making it suitable for training deep neural networks. It allows for parallelization of computations across multiple CPUs or GPUs, allowing for faster training times. TensorFlow also provides a high-level API called Keras that simplifies the process of building and training models. TensorFlow offers a wide range of tools and libraries that make it easy to integrate with other Python libraries and frameworks. It has built-in support for data preprocessing and visualization, making it easy to prepare data for training and analyze model performance. One of the major advantages of TensorFlow is its ability to deploy models to a variety of platforms, including mobile devices and the web.

- **Graph-based computation:** TensorFlow uses a graph-based computation model, which allows for efficient execution of computations across multiple devices and CPUs/GPUs.
- **Automatic differentiation:** TensorFlow provides automatic differentiation, which allows for efficient computation of gradients for use in backpropagation algorithms.
- **High-level APIs:** TensorFlow provides high-level APIs, such as Keras, that allow developers to quickly build and train complex models with minimal code.
- **Preprocessing and data augmentation:** TensorFlow provides a range of tools for preprocessing and data augmentation, including image and text preprocessing, data normalization, and more.

- **Distributed training:** TensorFlow supports distributed training across multiple devices, CPUs, and GPUs, allowing for faster training times and more efficient use of resources.
- **Model deployment:** TensorFlow allows for easy deployment of models to a variety of platforms, including mobile devices and the web.
- **Visualization tools:** TensorFlow provides a range of visualization tools for analyzing model performance, including TensorBoard, which allows for real-time visualization of model training and performance.

5.4 PYCHARM

PyCharm is an integrated development environment (IDE) for Python programming language, developed by JetBrains. PyCharm provides features such as code completion, debugging, code analysis, refactoring, version control integration, and more to help developers write, test, and debug their Python code efficiently.

PyCharm is available in two editions: Community Edition (CE) and Professional Edition (PE). The Community Edition is a free, open-source version of the IDE that provides basic functionality for Python development. The Professional Edition is a paid version of the IDE that provides advanced features such as remote development, web development, scientific tools, database tools, and more.

PyCharm is available for Windows, macOS, and Linux operating systems. It supports Python versions 2.7, 3.4, 3.5, 3.6, 3.7, 3.8, 3.9, and 3.10. PyCharm provides features such as code completion, debugging, code analysis, refactoring, version control integration, and more to help developers write, test, and debug their Python code efficiently. The Community Edition is a free, open-source version of the IDE that provides basic functionality.

Features:

- Intelligent code completion
- Syntax highlighting
- Code inspection
- Code navigation and search
- Debugging
- Testing
- Version control integration
- Web development support
- Scientific tools support
- Database tools support

Integration with other JetBrains tools

PyCharm's code completion feature can help speed up development by automatically suggesting code based on context and previously written code. It also includes a debugger that allows developers to step through code, set breakpoints, and inspect variables.

PyCharm has integration with version control systems like Git, Mercurial, and Subversion. It also supports virtual environments, which allow developers to manage different Python installations and packages in isolated environments.

The IDE also has features specifically geared towards web development, such as support for popular web frameworks like Django, Flask, and Pyramid. It includes tools for debugging, testing, and profiling web applications

Customization:

PyCharm allows developers to customize the IDE to their liking. Users can change the color scheme, fonts, and other settings to make the IDE more comfortable to use.

Collaboration:

PyCharm makes it easy for developers to collaborate on projects. It supports integration with popular collaboration tools such as GitHub, Bitbucket, and GitLab. It also includes features for code reviews, task management, and team communication.

Education:

PyCharm provides a learning environment for Python programming language. PyCharm Edu is a free, open-source edition of PyCharm that includes interactive courses and tutorials for learning Python. It provides an easy-to-use interface for beginners and includes features such as code highlighting, autocompletion, and error highlighting.

Support:

PyCharm has an active community of users who provide support through forums and social media. JetBrains also provides comprehensive documentation, tutorials, and training courses for PyCharm.

For users who need more personalized support, JetBrains offers a paid support plan that includes email and phone support.

Pricing:

PyCharm Community Edition is free and open-source. PyCharm Professional Edition requires a paid license, but offers a 30-day free trial.

Integrations:

PyCharm integrates with a wide range of tools and technologies commonly used in Python development. It supports popular Python web frameworks like Flask, Django, Pyramid, and web2py.

Performance:

PyCharm is known for its fast and reliable performance. It uses a combination of static analysis, incremental compilation, and intelligent caching to provide fast code completion and navigation. PyCharm also has a memory profiler that helps identify and optimize memory usage in Python applications.

Ease of Use:

PyCharm provides an intuitive and easy-to-use interface for developers. It has a well-organized menu structure, clear icons, and easy-to-navigate tabs. PyCharm also provides a variety of keyboard shortcuts and customizable keymaps that allow users to work efficiently without constantly switching between the mouse and keyboard.

Community:

PyCharm has a large and active community of developers who contribute to the development of the IDE. The PyCharm Community Edition is open-source, which means that anyone can contribute to its development.

Database : SQL

MySQL is the world's most used open source relational database management system (RDBMS) as of 2008 that run as a server providing multi-user access to a number of databases. The MySQL development project has made its source code available under the terms of the GNU General Public License , as well as under a variety of proprietary agreements.

Inter images

MySQL is primarily an RDBMS and ships with no GUI tools to administer MySQL databases or manage data contained within the databases. Users may use the included command line tools, or use MySQL "front-ends", desktop software and web applications that create and manage MySQL databases, build database structures, back up data, inspect status, and work with data records.

Graphical

The official MySQL Workbench is a free integrated environment developed by MySQL AB, that enables users to graphically administer MySQL databases and visually design database structures. MySQL Workbench replaces the previous package of software, MySQL GUI Tools. Similar to other third-party packages, but still considered the authoritative MySQL frontend, MySQL Workbench lets users manage database design & modeling.

5.5 OPERATING SYSTEM

Windows 10 (64-bit) is a modern, reliable, and widely used operating system that supports a broad range of applications with high performance and stability. Its 64-bit architecture enables efficient utilization of system resources, including increased RAM capacity and enhanced processing power. This makes it particularly suitable for running advanced software applications, databases, and resource-intensive systems such as blockchain-based platforms and machine learning models. The operating system provides a user-friendly interface, robust security features, and compatibility with a wide range of development tools and software environments. It supports multitasking and efficient memory management, which are essential for developing and testing complex applications. Additionally, Windows 10 offers built-in security mechanisms such as Windows Defender, firewall protection, and regular updates to safeguard against vulnerabilities and cyber threats.

For software development, the system uses PyCharm as the primary Integrated Development Environment (IDE). PyCharm is a powerful and feature-rich IDE specifically designed for Python development. It provides seamless integration with various development tools and enhances productivity through its intelligent features. One of the key features of PyCharm is its intelligent code completion, which suggests relevant code snippets based on context, reducing development time and minimizing errors. The IDE also includes a built-in debugger, allowing developers to step through code execution, set breakpoints, and inspect variables efficiently, thereby simplifying the debugging process. In addition, PyCharm offers extensive support for web development frameworks such as Django, Flask, and Pyramid. It includes built-in tools for debugging, testing, and profiling web applications, making the development process more efficient and streamlined .

CHAPTER 6

SYSTEM DESIGN

6.1 SYSTEM ARCHITECTURE

The architecture of the proposed system consists of multiple integrated modules working together to ensure secure certificate generation, storage, and verification. The process begins with the user and institution interacting through the application interface, where certificates are created and authenticated using facial recognition based on the Grassmann algorithm. Once verified, the certificate data is encrypted using a hybrid encryption technique combining AES for data security and ECC for secure key management. The encrypted certificate is then stored in a decentralized blockchain network, ensuring immutability and tamper-proof storage.

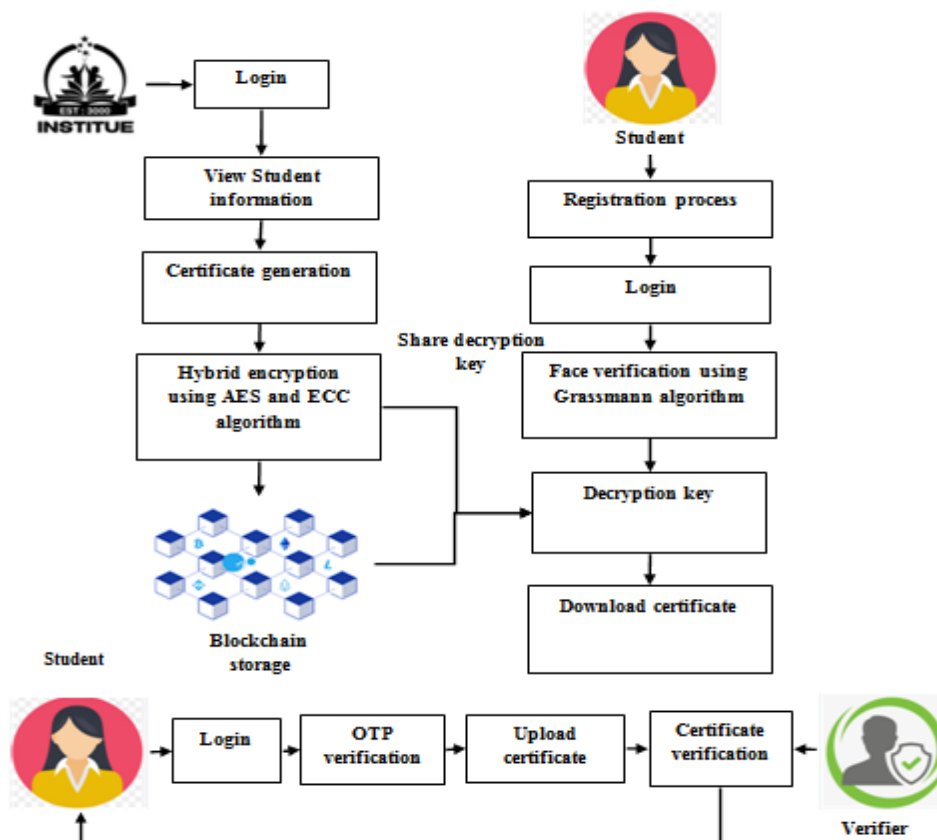


Fig. No. 6.1 System Architecture

6.2 UML DIAGRAMS

A UML (Unified Modeling Language) diagram is a standardized visual representation used to model, design, and document software systems. It provides a common language for developers, designers, and stakeholders to understand how different components of a system interact with each other. By representing complex systems in a graphical format, UML diagrams make it easier to analyze, design, and communicate system functionality effectively. UML diagrams are broadly classified into two main categories: structural diagrams and behavioral diagrams. Structural diagrams describe the static aspects of a system, focusing on how system components are organized. Examples include class diagrams, object diagrams, and component diagrams, which illustrate system architecture, data structures, and relationships between different elements. On the other hand, behavioral diagrams represent the dynamic aspects of a system, showing how components interact over time. These include use case diagrams, sequence diagrams, and activity diagrams, which help in understanding workflows, processes, and system behavior.

These diagrams act as blueprints for software development, covering various stages such as requirement analysis, system design, implementation, and maintenance. They ensure a clear and consistent understanding among team members, reducing ambiguity and improving collaboration. UML diagrams are particularly useful in identifying system requirements, defining system architecture, and visualizing interactions between different modules. For instance, a use case diagram illustrates the interaction between users (actors) and the system, highlighting the functionalities provided by the system. A class diagram represents the structure of the system by showing classes, attributes, methods, and relationships such as inheritance and association. A sequence diagram depicts the flow of messages between objects in a time sequence, helping to understand the control flow and logic of system operations.

6.2.1 USECASE DIAGRAM

A Use Case Diagram is a visual representation used in system design to illustrate how users interact with a system. It describes the functional requirements of a system by showing the relationship between actors (users or external entities) and use cases (system functionalities). This diagram helps in understanding the overall behavior of the system from the user's perspective.

In its most basic form, a use case diagram depicts the interaction between users and the system, highlighting how different functionalities are accessed and utilized. A “system” in this context refers to the application or platform being developed, such as a certificate verification system.

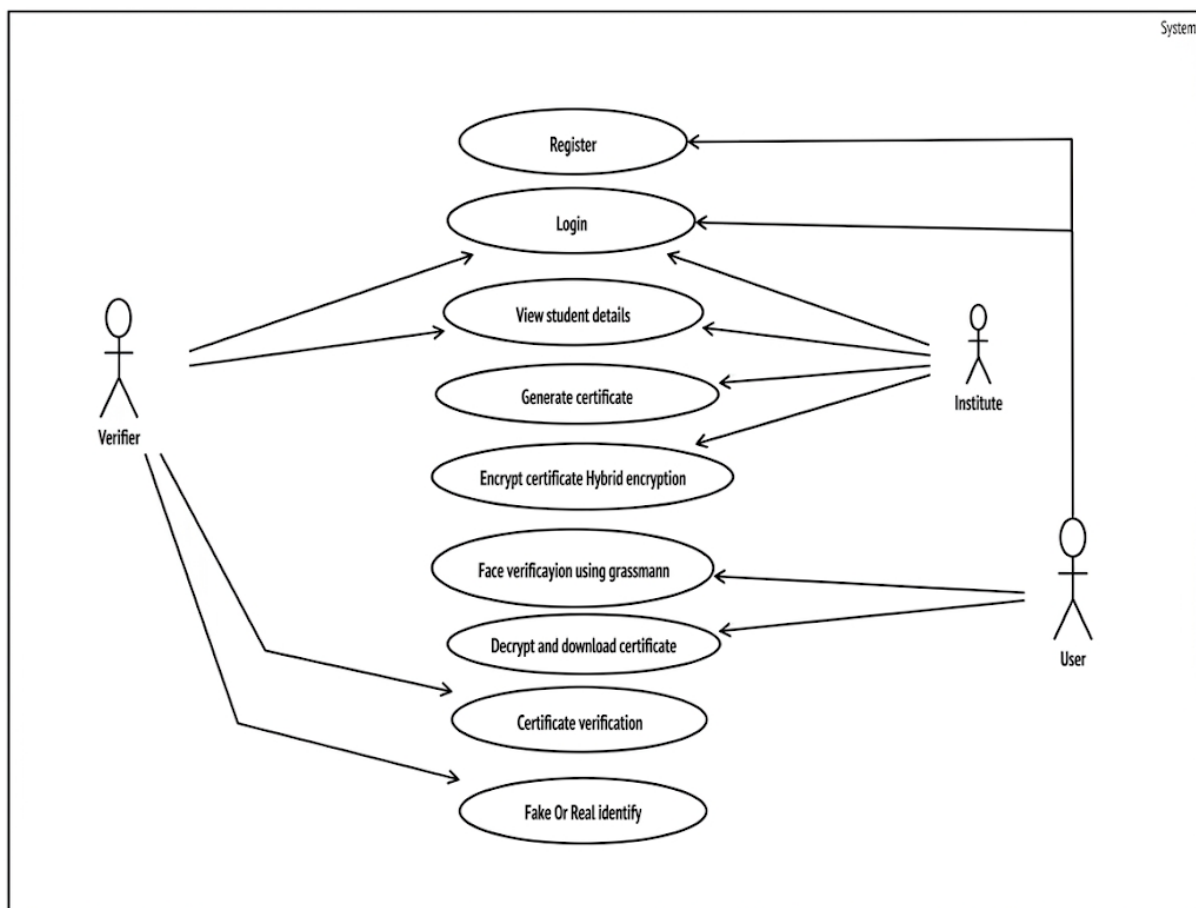


Fig. No. 6.2 Use Case Diagram

6.2.2 CLASS DIAGRAM

A class diagram, as defined by the Unified Modeling Language (UML), is a kind of static structural diagram that illustrates a system's classes, properties, functions, and interactions between objects.

The fundamental component of object-oriented modeling is the class diagram. It is utilized for both technical modeling which converts the models into computer code and general conceptual modeling of the applications systematic. It provides a blueprint of the system by showing how different components are organized and how they interact with each other.

The class diagram is a fundamental component of object-oriented modeling, as it helps in designing and understanding the architecture of a software system.

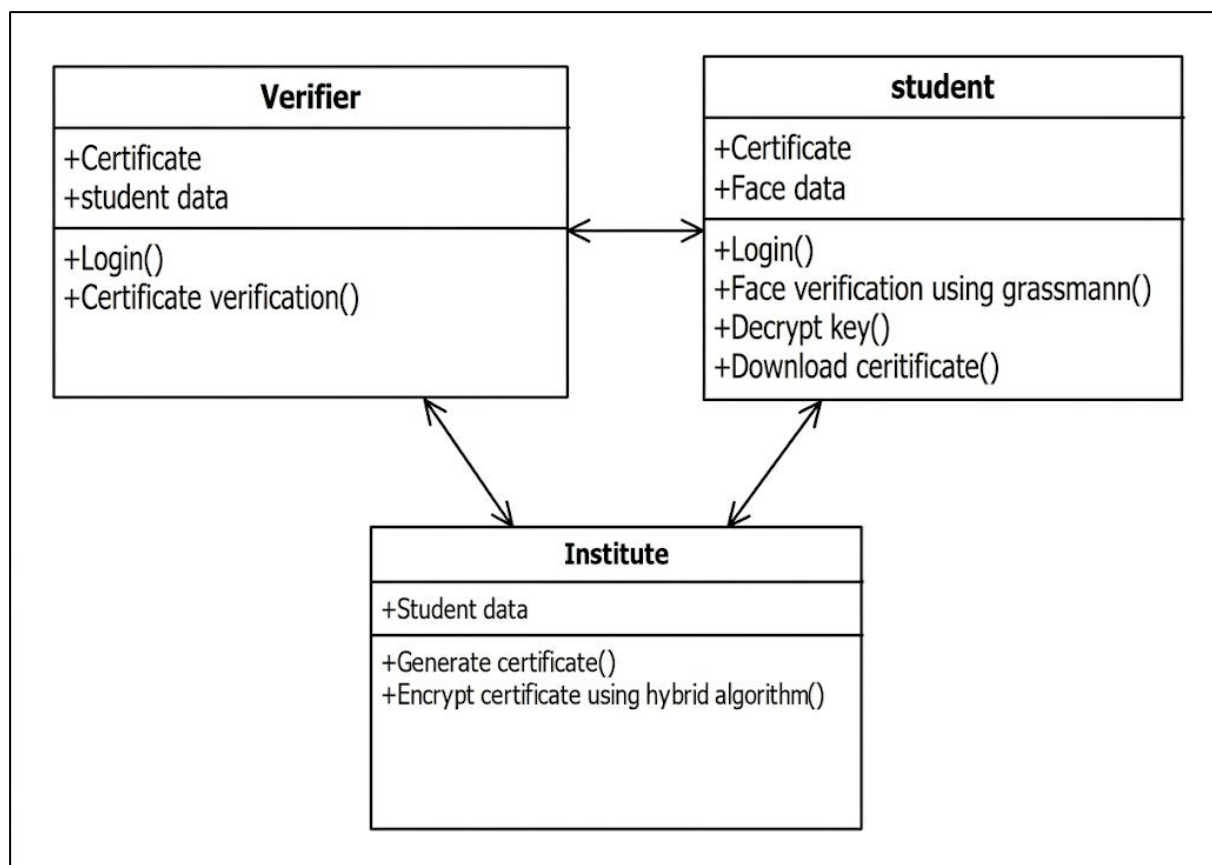


Fig. No. 6.3 Class Diagram

6.2.3 SEQUENCE DIAGRAM

An object's interactions are arranged chronologically in a sequence diagram. It shows the classes and objects that are a part of the scenario as well as the messages that are passed between the objects in order for the scenario to work.

Sequence diagrams are commonly linked to the realizations of use cases in the Logical View of the system that is being developed. Event diagrams or event scenarios are other names for sequence diagrams.

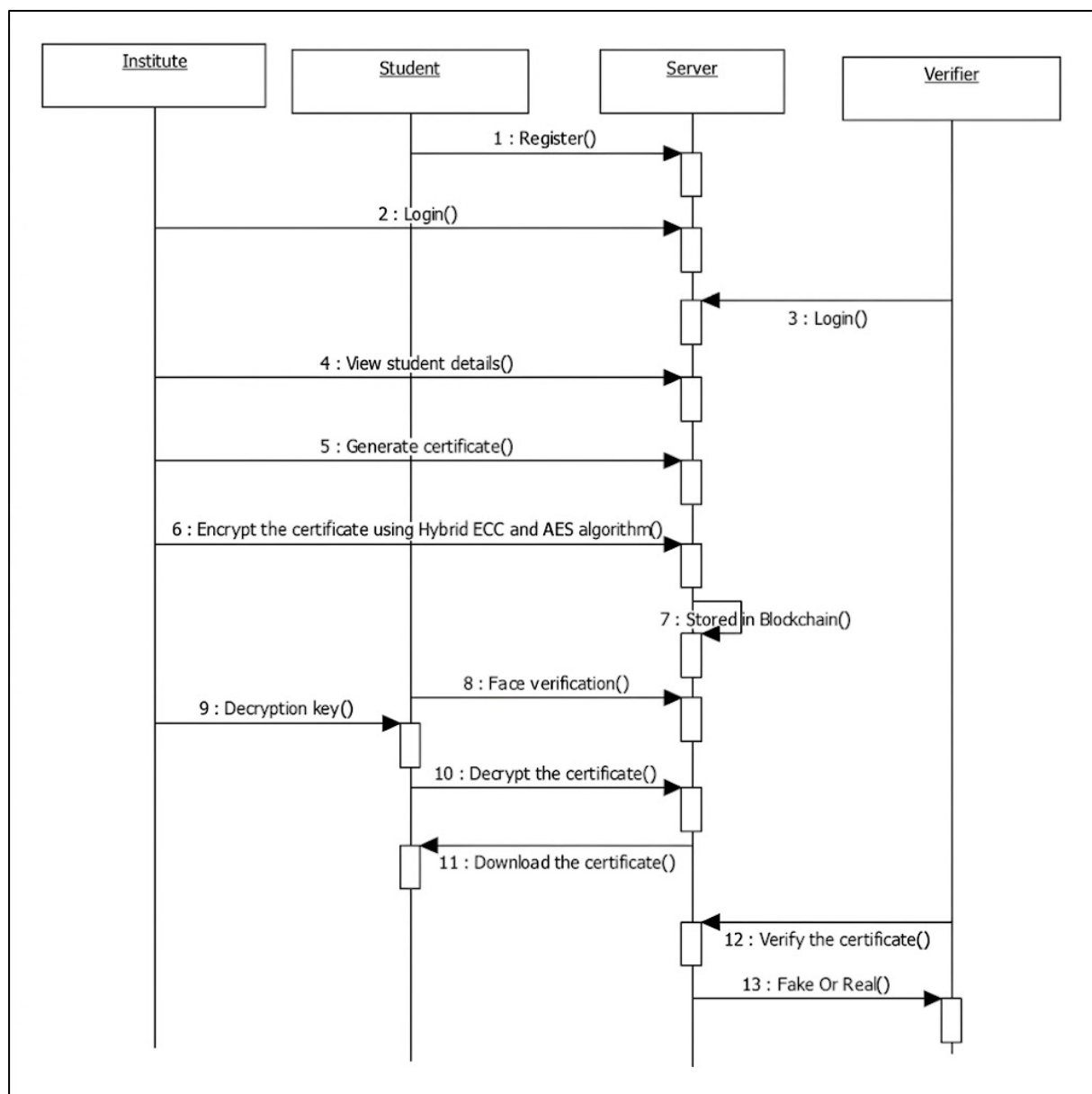


Fig. No. 6.4 Sequence Diagram

6.2.4 DEPLOYMENT DIAGRAM

A Deployment Diagram is a type of Unified Modeling Language (UML) diagram used to represent the physical architecture of a system. It visually illustrates how software components are deployed on hardware nodes and how different parts of the system communicate with each other. This diagram helps in understanding the relationship between hardware and software in a real-world environment.

Typically, deployment diagrams are used to visually represent to consumers the link between hardware and software. In software development, a typical deployment diagram consists of two primary components: artifacts, which are often client or database schema, and nodes, which are essentially various types of servers. The Artifacts are housed on Nodes, and various kinds of Artifacts function on Nodes.

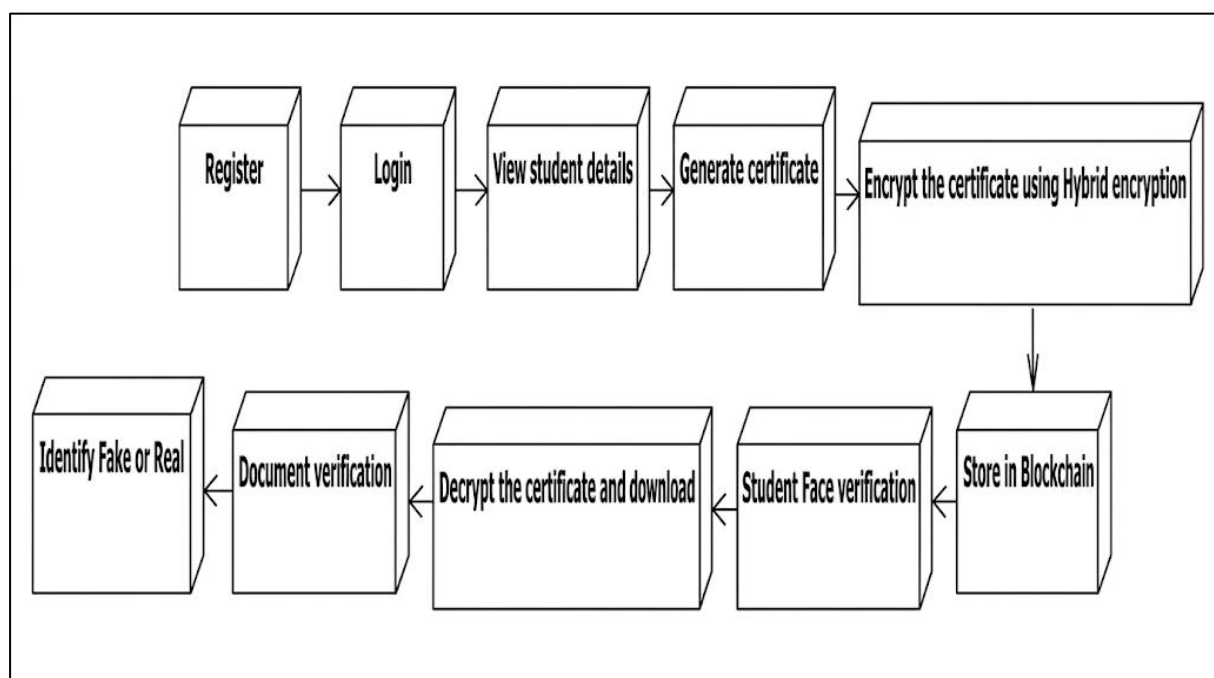


Fig. No. 6.5 Collaboration Diagram

6.2.5 DATA FLOW DIAGRAM

A two-dimensional diagram explains how data is processed and transferred in a system. The graphical depiction identifies each source of data and how it interacts with other data sources to reach a common output. Individuals seeking to draft a data flow diagram must identify external inputs and outputs, determine how the inputs and outputs relate to each other, and explain with graphics how these connections relate and what they result in. This type of diagram helps business development and design teams visualize how data is processed and identify or improve certain aspects.

LEVEL 0

The Level 0 DFD shows how the system is divided into 'sub-systems' (processes), each of which deals with one or more of the data flows to or from an external agent, and which together provide all of the functionality of the system as a whole. It also identifies internal data stores that must be present in order for the system to do its job, and shows the flow of data between the various parts of the system.

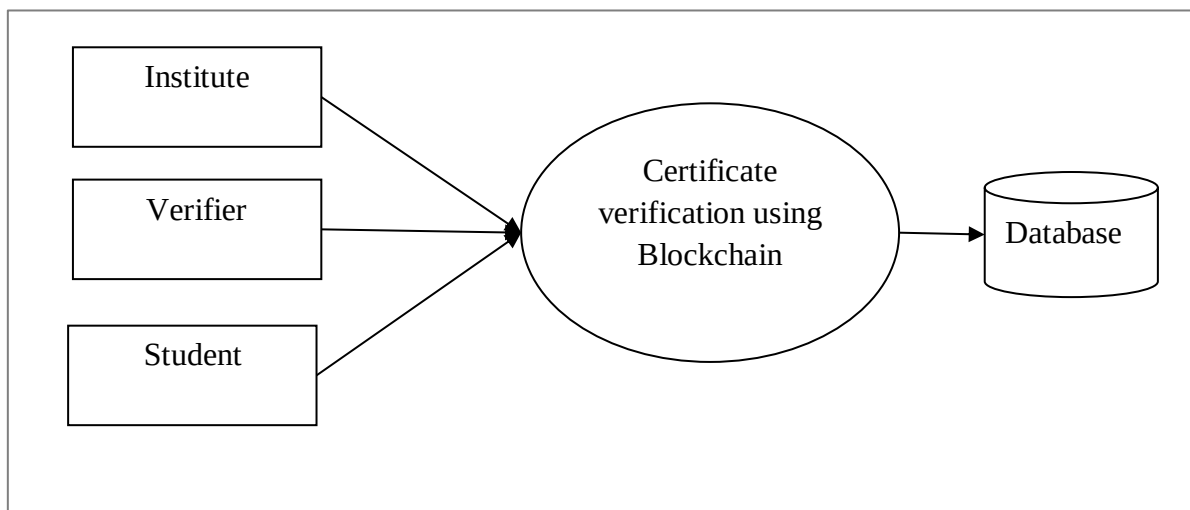


Fig. No. 6.6 Data flow diagram level 0

LEVEL-1

The next stage is to create the Level 1 Data Flow Diagram. This highlights the main functions carried out by the system.

As a rule, to describe the system was using between two and seven functions - two being a simple system and seven being a complicated system. This enables us to keep the model manageable on screen or paper.

The system is typically described using two to seven processes, where fewer processes indicate a simple system and more processes represent a complex system. This approach helps maintain clarity and ensures that the diagram remains understandable and manageable on paper or screen.

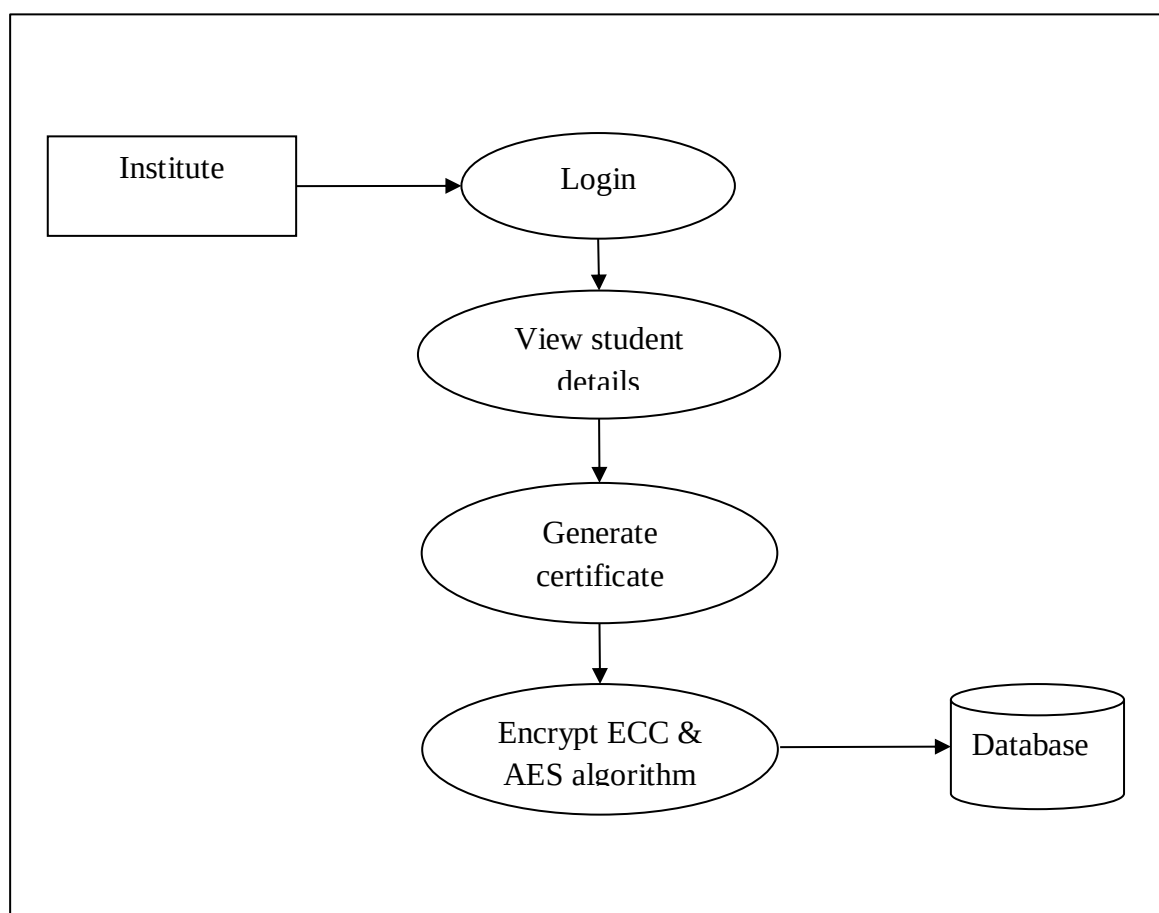


Fig. No. 6.7 Data flow diagram level 1

LEVEL-2

A Data Flow Diagram (DFD) tracks processes and their data paths within the business or system boundary under investigation. A DFD defines each domain boundary and illustrates the logical movement and transformation of data within the defined boundary.

The diagram shows 'what' input data enters the domain, 'what' logical processes the domain applies to that data, and 'what' output data leaves the domain. Essentially, a DFD is a tool for process modeling and one of the oldest.

A DFD defines the system boundary (domain) and shows the logical flow and transformation of data within that boundary. It focuses on what happens to the data, rather than how the system is physically implemented.

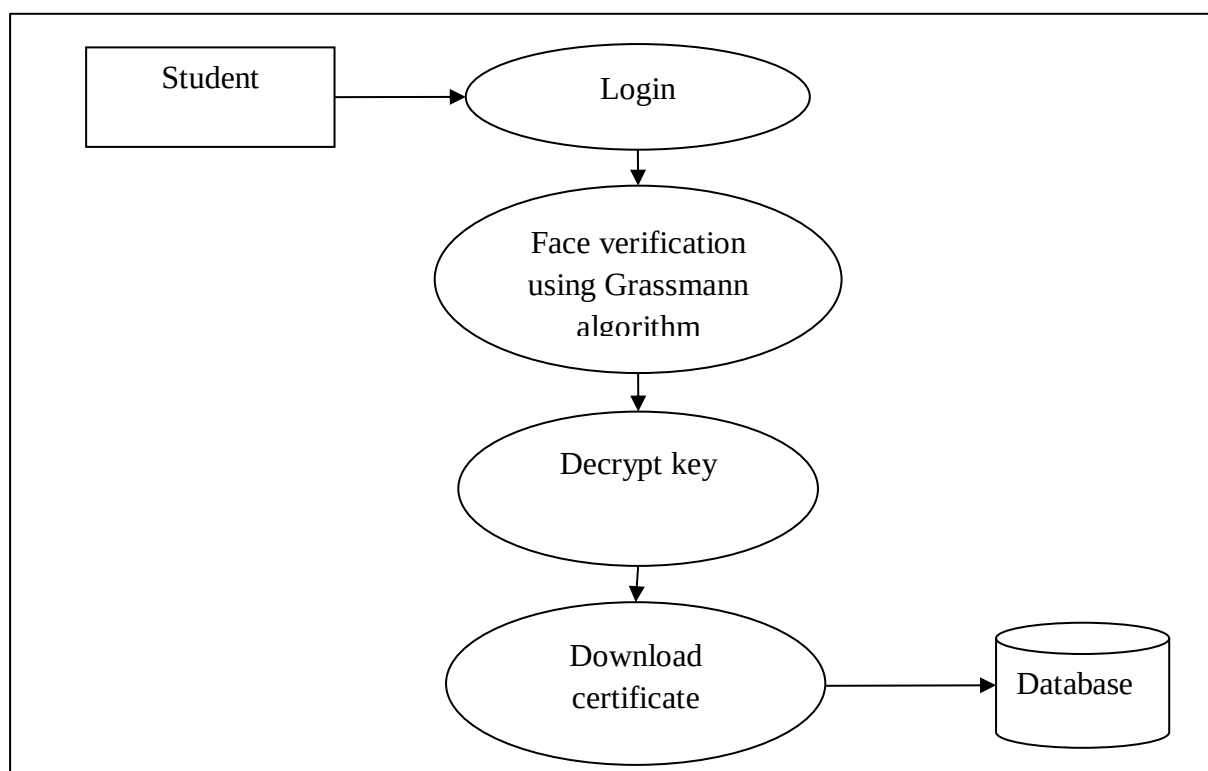


Fig. No. 6.8 Data flow diagram level 2

LEVEL-3

A data flow diagram (DFD) is a graphical representation of the flow of data through an information system. A DFD shows the flow of data from data sources and data stores to processes, and from processes to data stores and data sinks.

DFDs are used for modelling and analyzing the flow of data in data processing systems, and are usually accompanied by a data dictionary, an entity-relationship model, and a number of process descriptions.

A DFD is a graphical representation of how data flows through an information system. It illustrates the movement of data from data sources (external entities) to processes, between processes, and finally to data stores or data sinks (outputs). In Level–3, each process is broken down into detailed steps, showing the exact sequence of operations and how data is transformed at each stage.

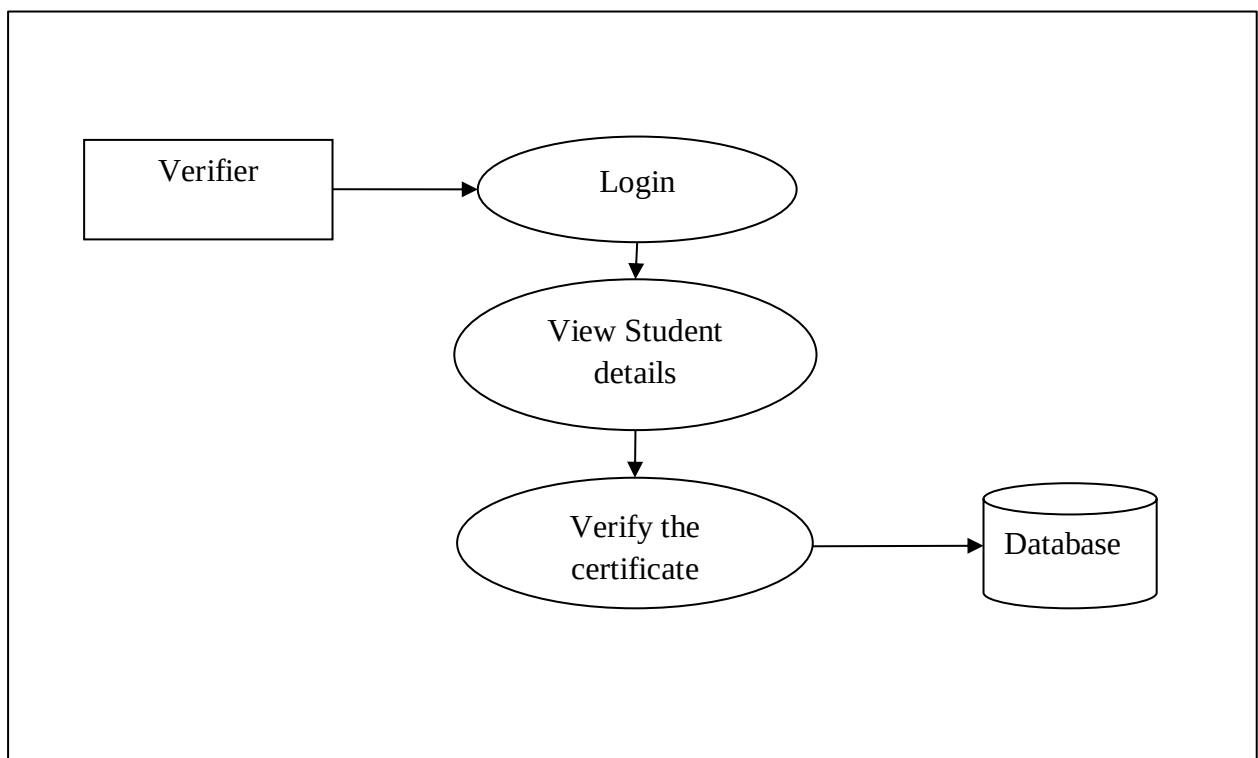


Fig. No. 6.9 Data flow diagram level 3

LEVEL 4

A DFD may look similar to a flow chart. However, there is a significant difference with the data flow diagram. The arrows in DFDs show that there is a flow of data between the two components and not that the component is sending the data that must be executed in the following component.

A component in DFD may not continue execution when sending data and during execution of the component receiving the data. The component sending data can send multiple sets of data along several connections. In fact, a DFD node can be a component that never ends.

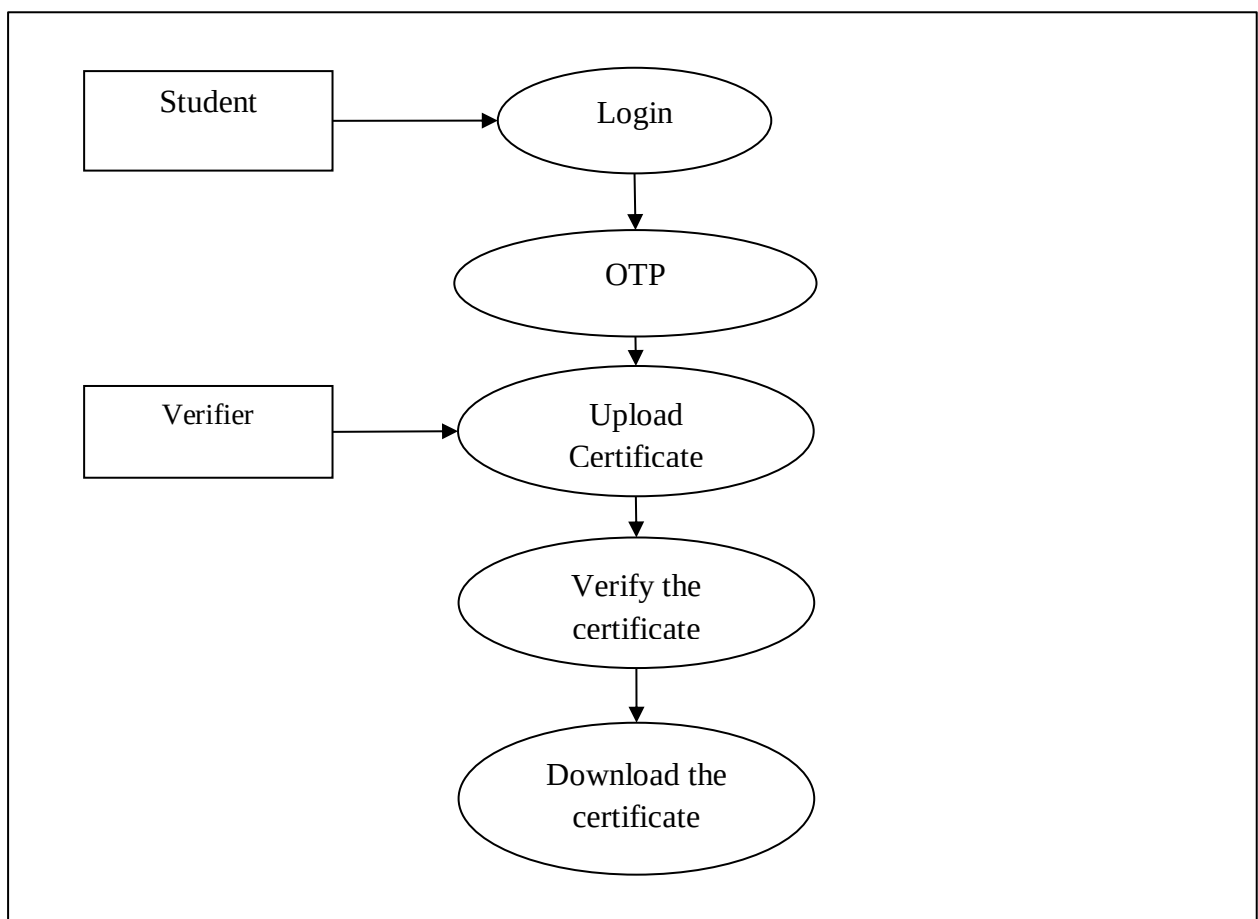


Fig. No. 6.10 Data flow diagram level 4

CHAPTER 7

PROJECT DESCRIPTION

7.1 DESCRIPTION

The project presents a secure and intelligent certificate verification system that integrates facial biometric authentication, hybrid encryption, and blockchain technology to ensure authenticity, confidentiality, and integrity of digital certificates. It uses the Grassmann algorithm for facial recognition to verify user identity, ensuring that only authorized individuals can access or manage certificates. The system applies a hybrid encryption approach combining AES and ECC to securely encrypt certificate data and protect cryptographic keys during storage and transmission. Blockchain technology is used to store certificate records in a decentralized and tamper-proof manner, preventing unauthorized modifications. Additionally, OTP and hash-based validation enhance security during verification, making the system a reliable, transparent, and efficient solution for modern digital certificate management.

7.2 MODULES DESCRIPTION

7.2.1 FRAMEWORK CREATION

The framework creation module defines the overall system architecture for secure certificate verification. It establishes the integration between user interface, backend processing, and blockchain network. This module designs the communication flow between all system components.

It initializes databases required for storing user, institution, and certificate data. Security layers are configured to manage authentication and authorization. API connections are established for interaction between modules. The system may use both traditional databases for operational data and blockchain storage for secure and immutable certificate records.

7.2.2 INSTITUTION PROCESS

The institution process module handles certificate creation and management by authorized organizations. Institutions log into the system using secure credentials. After authentication, they initiate certificate generation requests. User details and academic or identity information are entered into the system. The validity of the entered data is verified by institutional authorities. Once verified, the certificate is prepared for encryption. Facial authentication ensures the legitimacy of the issuing authority. The system binds certificate data with user identity securely. Metadata such as issue date and certificate ID are generated.

The certificate is forwarded for encryption processing. OTP-based validation confirms administrative actions. The module ensures only authorized institutions can issue certificates. It prevents unauthorized creation or modification of records.

7.2.3 FILE ENCRYPTION (HYBRID ALGORITHM)

The file encryption module secures certificate data using a hybrid cryptographic approach. It begins by receiving certificate files from the institution module. A symmetric AES key is generated for fast encryption. The certificate data is encrypted using AES encryption standards. AES ensures confidentiality and efficiency in processing large data. After encryption, the AES key is protected using ECC encryption.

ECC provides secure key exchange and asymmetric security. The encrypted AES key is stored separately from the data. Both encrypted file and key are combined for transmission. This ensures dual-layer security for sensitive information. Even if intercepted, data remains unreadable without keys. Key generation follows secure cryptographic standards. Randomization techniques are applied to strengthen encryption. The module ensures data integrity during encryption process.

7.2.4 BLOCKCHAIN STORAGE

The blockchain storage module ensures decentralized and tamper-proof certificate storage. It receives encrypted certificate data from the encryption module. A unique hash is generated for each certificate. The hash acts as a digital fingerprint of the certificate. A blockchain transaction is created containing certificate metadata. The transaction includes timestamp, certificate ID, and hash value. The data is validated using blockchain consensus mechanisms. Once validated, it is added to a new block. The block is linked to previous blocks forming a chain.

7.2.5 USER AUTHENTICATION

The user authentication module verifies the identity of users accessing the system. It begins with user login through secure credentials. Facial image capture is performed using camera input. The system extracts facial features using the Grassmann algorithm. These features are compared with stored templates. A similarity score is calculated for verification. If the match is successful, access is granted. An OTP is generated for second-level authentication. The OTP is sent to registered mobile or email. The user must enter OTP for validation.

Hash verification is performed for additional security. Multi-factor authentication ensures strong identity protection. Unauthorized users are blocked from system access.

7.2.6 DECRYPTION AND DOWNLOAD FILE

The decryption module enables authorized users to retrieve original certificates securely. After authentication, the system fetches encrypted certificate data. The corresponding encrypted AES key is also retrieved. ECC private key is used to decrypt the AES key. Once the AES key is recovered, file stored hash is compared with input certificate hash. If both match, authenticity is confirmed.

7.2.7 VERIFIER PROCESS

The verifier process module allows third parties to validate certificate authenticity. The verifier logs into the system with authorized access. Certificate data or hash is provided for verification. The system retrieves corresponding blockchain records. The stored hash is compared with input certificate hash. If both match, authenticity is confirmed.

If mismatch occurs, certificate is flagged as invalid. Blockchain ensures tamper-proof validation records. Verification does not require access to original data. OTP may be used for additional verifier authentication.

7.3 ALGORITHMS

7.3.1 GRASSMANN ALGORITHM (FACIAL RECOGNITION)

The Grassmann algorithm is used in the proposed system for robust facial biometric authentication.

It works by representing facial images in a high-dimensional subspace and mapping them onto a Grassmann manifold for efficient comparison. The system begins by capturing the user's facial image and preprocessing it through normalization, resizing, and noise removal.

Feature extraction is then performed to identify key facial patterns such as edges, contours, and spatial relationships. These extracted features are converted into mathematical subspaces that allow better discrimination between different individuals even under variations in lighting, pose, or expression. Once the facial features are mapped, the algorithm compares the query face with stored reference templates using similarity measures on the Grassmann manifold.

This approach improves accuracy in identity verification by focusing on geometric relationships rather than raw pixel values. If the similarity score exceeds a defined threshold, the user is authenticated successfully; otherwise, access is denied.

7.3.2 AES (ADVANCED ENCRYPTION STANDARD) – SYMMETRIC ENCRYPTION

AES is used in the system to securely encrypt certificate data before storage or transmission. It is a symmetric encryption algorithm where the same secret key is used for both encryption and decryption. In the proposed system, once the certificate is generated, it is converted into ciphertext using AES encryption rounds, which include SubBytes, ShiftRows, MixColumns, and AddRoundKey operations. These transformations ensure that the original data is completely concealed and cannot be interpreted without the correct key.

7.3.3 ECC (ELLIPTIC CURVE CRYPTOGRAPHY) – ASYMMETRIC ENCRYPTION

ECC is used in the system to securely manage and protect encryption keys used in AES encryption. It is an asymmetric cryptographic technique that uses a public-private key pair. After AES encryption of certificate data, the AES key itself is encrypted using ECC public key encryption. This ensures that even if the encrypted data is intercepted, the key required for decryption remains secure and inaccessible without the private key.

7.3.4 BLOCKCHAIN TRANSACTION AND STORAGE ALGORITHM

The blockchain algorithm is used to store certificate records in a decentralized and tamper-proof manner. Once a certificate is encrypted, a unique hash value is generated to represent its identity. This hash, along with metadata such as timestamp, certificate ID, and issuer details, is packaged into a blockchain transaction.

The process begins after a certificate is successfully authenticated and encrypted. A unique cryptographic hash value is generated from the certificate data using a secure hashing algorithm (such as SHA-256). This hash acts as a digital fingerprint that uniquely represents the certificate. Even a minor change in the certificate data will produce a completely different hash value, making any tampering easily detectable.

CHAPTER 8

SYSTEM TESTING

8.1 TESTING OBJECTIVES

Testing can never completely identify all the defects within software. Instead, it furnishes a criticism or comparison that compares the state and behaviour of the product against oracles principles or mechanisms by which someone might recognize a problem. These oracles may include (but are not limited to) specifications, contracts, comparable products, past versions of the same product, inferences about intended or expected purpose, user or customer expectations, relevant standards, applicable laws, or other criteria.

A primary purpose of testing is to detect software failures so that defects may be discovered and corrected. Testing cannot establish that a product functions properly under all conditions but can only establish that it does not function properly under specific conditions. The scope of software testing often includes examination of code as well as execution of that code in various environments and conditions as well as examining the aspects of code: does it do what it is supposed to do and do what it needs to do. In the current culture of software development, a testing organization may be separate from the development team. There are various roles for testing team members. Information derived from software testing may be used to correct the process by which software is developed.

Every software product has a target audience. For example, the audience for video game software is completely different from banking software. Therefore, when an organization develops or otherwise invests in a software product, it can assess whether the software product will be acceptable to its end users, its target audience, its purchasers, and other stakeholders. Software testing is the process of attempting to make this assessment

8.2 TYPES OF TESTING

8.2.1 Unit Testing

Unit testing verifies individual components or functions of software at the code level. It is usually performed by developers to ensure each unit works correctly.

It focuses on testing constructors, methods, and logic independently. Multiple test cases are used to cover different conditions, ensuring that each building block of the software functions properly.

8.2.2 Integration Testing

Integration testing checks how different modules or components of software interact with each other. It ensures that interfaces between integrated parts function correctly.

8.2.3 Performance Testing

Performance testing evaluates system behavior under different workloads to measure speed, responsiveness, and stability. It includes load, stress, and volume testing to analyze scalability and reliability.

8.2.4 Security Testing

Security testing is performed to identify vulnerabilities and protect software from unauthorized access or cyber attacks. It ensures that confidential data is secure and the system is protected from threats.

8.2.5 Usability Testing

Usability testing evaluates how easy and user-friendly the software interface is for users. It focuses on user experience, ensuring that the application is simple to understand and operate. This testing checks whether the interface works as intended and meets user expectations, improving overall satisfaction and usability of the system. This testing examines whether the interface works as intended and meets user expectations.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENT

9.1 CONCLUSION

The proposed certificate verification system successfully addresses the critical issues of forgery, data manipulation, and unauthorized access in traditional and centralized verification methods. By integrating facial biometric authentication using the Grassmann algorithm, the system ensures that only legitimate users can access and manage certificate-related operations. This significantly enhances identity assurance and reduces the risks of impersonation and fraudulent certificate usage in academic and professional environments.

Furthermore, the combination of hybrid encryption techniques (AES and ECC) provides a strong and efficient security framework for protecting sensitive certificate data during storage and transmission. AES ensures fast and reliable encryption of data, while ECC secures encryption keys, offering a dual-layer protection mechanism. In addition, blockchain technology adds an immutable and decentralized storage layer, ensuring that all certificate records remain transparent, tamper-proof, and traceable, thereby eliminating the risks associated with centralized databases.

Overall, the integration of AI-based facial recognition, advanced cryptographic techniques, and blockchain technology results in a secure, scalable, and trustworthy certificate verification system. The inclusion of OTP and hash-based validation further strengthens the system by adding multi-factor authentication and integrity verification. This comprehensive approach enhances trust, efficiency, and security in digital certificate management, making it suitable for real-world deployment across educational institutions, government agencies, and corporate organizations.

9.2 FUTURE ENHANCEMENT

The future enhancement of the proposed certificate verification system focuses on improving scalability and performance to support large-scale deployment across national and global institutions. As the number of users and certificate records increases, optimizing blockchain transaction speed and storage efficiency will become essential. Future versions of the system can integrate advanced consensus mechanisms such as Proof of Stake (PoS) or hybrid blockchain models to reduce processing time and energy consumption while maintaining high security and decentralization.

Another important enhancement is the integration of more advanced artificial intelligence and deep learning models to improve facial recognition accuracy and robustness. Future work can replace or augment the Grassmann-based approach with modern deep learning architectures such as CNNs or transformer-based vision models to handle complex real-world scenarios like aging effects, low lighting, and partial occlusion. Additionally, multimodal biometric authentication (face, voice, and fingerprint) can be introduced to further strengthen identity verification and reduce false acceptance or rejection rates.

Furthermore, the system can be enhanced by incorporating interoperability with national digital identity frameworks and government databases for seamless certificate validation across multiple platforms. Cloud-based deployment with edge computing support can also be introduced to improve accessibility and reduce latency in remote regions. In addition, implementing smart contracts within the blockchain can automate certificate issuance, renewal, and revocation processes, making the system more intelligent, autonomous, and efficient in future digital ecosystems. Edge computing allows data processing closer to the user, reducing latency and improving performance, especially in remote or low-connectivity regions. This ensures faster authentication and verification processes while maintaining system efficiency.

CHAPTER 10

APPENDIX

10.1 SOURCE CODE

```
from flask import Flask, render_template, request, redirect, url_for, session,
send_file, flash
import mysql.connector
from ecies.utils import generate_key
# from ecies import encrypt, decrypt
import base64, os
import base64, os, sys
app = Flask(__name__)
app.secret_key = 'a'
@app.route('/')
def home():
return render_template('index.html')
@app.route('/CollegeLogin')
def CollegeLogin():
return render_template('CollegeLogin.html')
@app.route('/VerifierLogin')
def VerifierLogin():
return render_template('VerifierLogin.html')
@app.route('/UserLogin')
def UserLogin():
return render_template('UserLogin.html')
@app.route('/NewVerifier')
def NewVerifier():
return render_template('NewVerifier.html')
```

```

@app.route('/NewUser')
def NewUser():
    return render_template('NewUser.html')

@app.route("/serverlogin", methods=['GET', 'POST'])
def serverlogin():
    if request.method == 'POST':
        if request.form['uname'] == 'admin' and request.form['password'] == 'admin':
            conn = mysql.connector.connect(user='root', password="",
            host='localhost', database='1documentverificationaesdb')
            cur = conn.cursor()
            cur.execute("SELECT * FROM verifiertb ")
            data = cur.fetchall()
        ()
        cur.execute("SELECT * FROM verifiertb ")
        data = cur.fetchall()
    return render_template('CollegeHome.html', data=data)

@app..fetchall()
return render_template('SUserInfo.html', data=data)

@app.route("/GApplyInfo")
def GApplyInfo():
    conn = mysql.connector.connect(user='root', password="", host='localhost',
    database='1documentverificationaesdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM applytb where status='waiting'")
    data = cur.fetchall()
    conn = mysql.connector.connect(user='root', password="", host='localhost',
    database='1documentverificationaesdb')
    cur = conn.cursor()return hmac.new(byte_key, message,
    hashlib.sha256).hexdigest().upper()

```

```

import pyAesCrypt
import random
import string
# Read the file in chunks to avoid memory issues with large files
with open(file_path, "rb") as file:
    chunk = file.read(8192)
while chunk:
    hash_func.update(chunk)
    chunk = file.read(8192)
# Return the hex digest of the hash
return hash_func.hexdigest()
@app.route("/gpload", methods=['GET', 'POST'])
def gpload():
    if request.method == 'POST':
        file_hash = ""
        dno = request.form['dno']
        file = request.files['file']
import random
        fnew = random.randint(111, 999)
        savename = str(fnew) + file.filename
        file.save("static/upload/" + savename)
        file_path = "static/upload/" + savename
    try:
        file_hash = generate_file_hash(file_path)
    print(f"The hash of the file is: {file_hash}")
    except Exception as e:
    print(f"Error: {e}")
        hash2 = file_hash
        secp_k = generate_key()

```

```

privhex = secp_k.to_hex()
    pubhex = secp_k.public_key.format(True).hex()
    filepath = "./static/upload/" + savename
    head, tail = os.path.split(filepath)
    newfilepath1 = './static/Encrypt/' + str(tail)
# newfilepath2 = './static/Decrypt/' + str(tail)
encrypt(pubhex, filepath, newfilepath1)
print("Public_key:", pubhex)
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1documentverificationaesdb')
    cursor = conn.cursor()
    cursor.execute("SELECT * from applytb where id='" + session['did'] + "'")
    data22 = cursor.fetchone()
if data22:
    mail = data22[3]
    cursor = conn.cursor()
    cursor.execute(
"update applytb set DNo='" + dno + "',dfile='" + savename + "',Hash3='" +
hash2 + "',Enkey='" + pubhex + "',status='upload' where id='" +
    session[
'did'] + "'")
    conn.commit()
    conn.close()
    mes = "Document No:" + dno + "\n" + "Decrypt Key: " + pubhex
    sendmail(mail, mes)
    flash('Document Upload Successfully ')
return GApplyInfo()
else:
    flash('Already Enter This DocumentNo..!')

```

```

return GApplyInfo()
"INSERT INTO veriortb VALUES ("," + uname + "," + mobile + "," +
email + "," + address + "," + username + "," + password + ")")
    conn.commit()
    conn.close()
    flash('Record Saved!')
return render_template('NewVerifier.html')
else:
    flash('Already Register This UserName!')
return render_template('NewVerifier.html')
@app.route("/ownerlogin", methods=['GET', 'POST'])
def ownerlogin():
if request.method == 'POST':
    username = request.form['uname']
    password = request.form['password']
    session['vname'] = request.form['uname']
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1documentverificationaesdb')
    cursor = conn.cursor()
    cursor. password="", host='localhost',
data = cur.fetchall()
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1documentverificationaesdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM verifytb where status !='waiting' and
VName='" + session['vname'] + "' ")
    data1 = cur.fetchall()
return render_template('VerifyDocument.html', data=data, data1=data1)
@app.route("/verify")

```

```

def verify():
    file_hash = "
    uname = "
    did = request.args.get('did')
    conn = mysql.connector.connect(user='root', password="", host='localhost',
    database='1documentverificationaesdb')
    cursor = conn.cursor()
    cursor.execute("SELECT * from verifytb where id ='" + str(did) + "' ")
    data = cursor.fetchone()
    if data:
        uname = data[1]
        dno = data[4]
        dfile = data[5]
        file_path = "static/uupload/" + dfile
    try:
        file_hash = generate_file_hash(file_path)
    print(f"The hash of the file is: {file_hash}")
    except Exception as e:
    print(f"Error: {e}")
        conn.close()        sendmail("velanguru372@gmail.com", uname + " User
    Upload Fake Document")
    return VerifyDocument()
@app.route("/vdownload")
def vdownload():
    vfid = request.args.get('vfid')
    st = request.args.get('st')
    if st != "orginal":
        flash('Fake Document!')
    return VerifyDocument()

```

```

else:
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1documentverificationaesdb')
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM verifytb where id='" + vfid + "'")
    data = cursor.fetchone()
if data:
    fname = data[5]
else:
    return 'Incorrect username / password !'
newfilepath2 = './static/uupload/' + str(fname)
return render_template('NewUser.html')
    cursor.execute("SELECT * from regtb where username='" + username + "'
and Password='" + password + "' ")
    data = cursor.fetchone()
if data is None:
    flash('Username or Password is wrong')
return render_template('UserLogin.html')
else:
    conn = mysql.connector.connect(user='root', password="",
host='localhost', database='1documentverificationaesdb')
    cursor = conn.cursor()
    cursor.execute("truncate table temptb")
    conn.commit()
    if data:
        Email = data[3]
        Phone = data[2]
    else:
        return 'Incorrect username / password !'

```

```

return uname, Email, Phone

    cursor.execute("SELECT * from temptb where UserName='" + uname + "' ")
    data = cursor.fetchone()
if data is None:
    flash('Face is wrong')
return render_template('UserLogin.html')
else:
return render_template('OTP.html')
@app.route("/votp", methods=['GET', 'POST'])
def votp():
if request.method == 'POST':
    otp = request.form['OTP']
@app.route('/Apply')
def Apply():
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1documentverificationaesdb')
    cur = conn.cursor()
    cur.execute("SELECT *          num1 = random.randrange(1111, 9999)
                hash2 =
create_sha256_signature("E49756B4C8FAB4E48222A3E7F3B97CC3",
str(num1))
                conn = mysql.connector.connect(user='root', password="",
host='localhost', database='1documentverificationaesdb')
                cursor = conn.cursor()
                cursor.execute(
"INSERT INTO applytb VALUES ('" + uname + "','" + mobile + "','" + email
+ "','" + address + "','" +
                username + "','" + dname + "','','','waiting','" + hash1 + "','" +
hash2 + "')"")

```

```

        conn.commit()
        conn.close()
        flash('Record Saved!')
return Apply()
else:
    hash1 = '0'
num1 = random.randrange(1111, 9999)
    hash2 =
create_sha256_signature("E49756B4C8FAB4E48222A3E7F3B97CC3",
str(num1))
        conn = mysql.connector.connect(user='root', password="",
host='localhost', database='1documentverificationaesdb')
        cursor = conn.cursor()
        cursor.execute(
"INSERT INTO applytb VALUES ("," + uname + "," + mobile + "," + email
+ "," + address + "," +
        username + "," + dname + "," + "," + 'waiting'," + hash1 + "," + hash2
+ ")")
        conn.commit()
        conn.close()
        flash('Record Saved!')
return Apply()
@app.route('/UDocumentInfo')
def UDocumentInfo():
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database='1documentverificationaesdb')
    cur = conn.cursor()
    cur.execute("SELECT * FROM applytb where username='" +
session['uname'] + "'")

```

```

    data1 = cur.fetchall()
return render_template('UDocumentInfo.html', data=data1)
@app.route("/userdownload")
def userdownload():
    ufid = request.args.get('ufid')
    st = request.args.get('st')
    session["ufid"] = ufid
if st == "waiting":
    flash('Document Request is waiting!')
    conn = mysql.connector.connect(user='root', password="", host='localhost',
database=return 'Incorrect username / password !'
if sear == prkey:
    filepath = "./static/Encrypt/" + fname
print(filepath)
    head, tail = os.path.split(filepath)
")
# Converts the Multipart msg into a string
text = msg.as_string()
# sending the mail
s.sendmail(fromaddr, toaddr, text)
# terminating the session
s.quit()
if __name__ == '__main__':
    app.run(host='0.0.0.0', debug=True, port=5000)
#app.run(debug=True, use_reloader=True)
LiveRecognition:
from __future__ import print_function
import sys, fsdk, math, ctypes, time
from fsdk import FSDK

```

```

license_key =
"fVrFCzYC5wOtEVspKM/zfLWVcSIZA4RNqx74s+QngdvRiCC7z7MHlSf2w
3+OUyAZkTFeD4kSpfVPcRVIqAKWUZzJG975b/P4HNNzpl11edXGIyGrTO
/DImoZksDSRs6wktvgr8lnNCB5IukIPV5j/jBKlgL5aqiwSfyCR8UdC9s="
if not fsdk.windows:
print('The program is for Microsoft Windows. '); exit(1)
import win
trackerMemoryFile = "tracker70.dat"
FONT_SIZE = 30
print("Initializing FSDK... ", end="")
FSDK.ActivateLibrary(license_key);
FSDK.Initialize()
print("OK\nLicense info:", FSDK.GetLicenseInfo())
FSDK.InitializeCapturing()
print('Looking for video cameras... ', end = "")
camList = FSDK.ListCameraNames()
if not camList: print("Please attach a camera.");
print(camList[0]) # camList[0].devicePath
camera = camList[0] # choose the first camera (0)
print("using '%s'" % camera)
formatList = FSDK.ListVideoFormats(camera)
#print(*zip(range(len(formatList)), formatList), sep='\n')
print(*formatList[0:5], sep='\n')
if len(formatList)>5: print('...', len(formatList)-5, 'more formats (skipped)...')
vfmt = formatList[0] # choose the first format: vfmt.Width, vfmt.Height,
vfmt.BPP
print('Selcted camera format:', vfmt)
FSDK.SetVideoFormat(camera, vfmt)
print("Trying to open '%s'... " % camera, end = "")

```

```

camera = FSDK.OpenVideoCamera(camera)
print("OK", camera.handle)
try: else:
if message == win.WM_MOUSEMOVE:
    updateActiveFace()
return 1
if message == win.WM_LBUTTONDOWN:
if activeFace and capturedFace != activeFace:
    capturedFace = activeFace
    win.SetWindowText(inpBox, fsdkTracker.GetName(capturedFace))
    win.ShowWindow(inpBox, win.SW_SHOW)
    win.SetFocus(inpBox)
else:
    capturedFace = None
win.ShowWindow(inpBox, win.SW_HIDE)
return 1
return win.DefWindowProc(hWnd, message, win.WPARAM(wParam),
win.LPARAM(lParam))
# instance of MIMEMultipart
msg = MIMEMultipart()
# storing the senders email address
msg['From'] = fromaddr
# storing the receivers email address
msg['To'] = toaddr
# storing the subject
msg['Subject'] = "Alert"
# string to store the body of the mail
body = mmmmsg
# attach the body with the msg instance

```

```

msg.attach(MIMEText(
    conn = mysql.connector.connect(user='root',
password=", host='localhost', database='1documentverificationaesdb')
        cursor = conn.cursor()
        cursor.execute(
"insert into temptb values(", "" + ss + ", " + str(n) + ")")
        conn.commit()
        conn.close()
else:
print('No')
        outFileName = 'static/out.jpg'
bmp = win.Bitmap.FromHBITMAP(img.GetHBitmap())
        saimg = FSDK.Image(bmp.GetHBITMAP())
        saimg.SaveToFile(outFileName, quality=85)
import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.base import MIMEBase
from email import encoders
        fromaddr = "projectmailm@gmail.com"
toaddr = Email
# instance of MIMEMultipart
msg = MIMEMultipart()
# storing the senders email address
msg['From'] = fromaddr, 'octet-stream')
# To change the payload into encoded form
p.set_payload((attachment).read())
# encode into base64
encoders.encode_base64(p)

```

```

        p.add_header('Content-Disposition', "attachment; filename= %s" %
filename)
# attach the instance 'p' to instance 'msg'
msg.attach(p)
# creates SMTP session
s = smtplib.SMTP('smtp.gmail.com', 587)
# start TLS for security
s.starttls(), text)
# terminating the session
s.quit()
if face_id in faces:
    tracker.draw(surfGr, gpath, face_id) #
fsdkTracker.GetFacialFeatures(face_id)) # draw existing tracker
msg = win.MSG()
if win.PeekMessage(win.byref(msg), 0, 0, 0, win.PM_REMOVE):
    win.TranslateMessage(win.byref(msg))
    win.DispatchMessage(win.byref(msg))
if msg.message == win.WM_KEYDOWN and msg.wParam ==
win.VK_ESCAPE or need_to_exit: break
print("Please wait while saving Tracker memory... ", end="", flush=True)
fsdkTracker.SaveToFile(trackerMemoryFile)
win.ShowWindow(hwnd, win.SW_HIDE)
img.Free()
fsdkTracker.Free()
camera.Close()
FSDK.FinalizeCapturing()
FSDK.Finalize()

```

10.2 SCREEN SHOTS

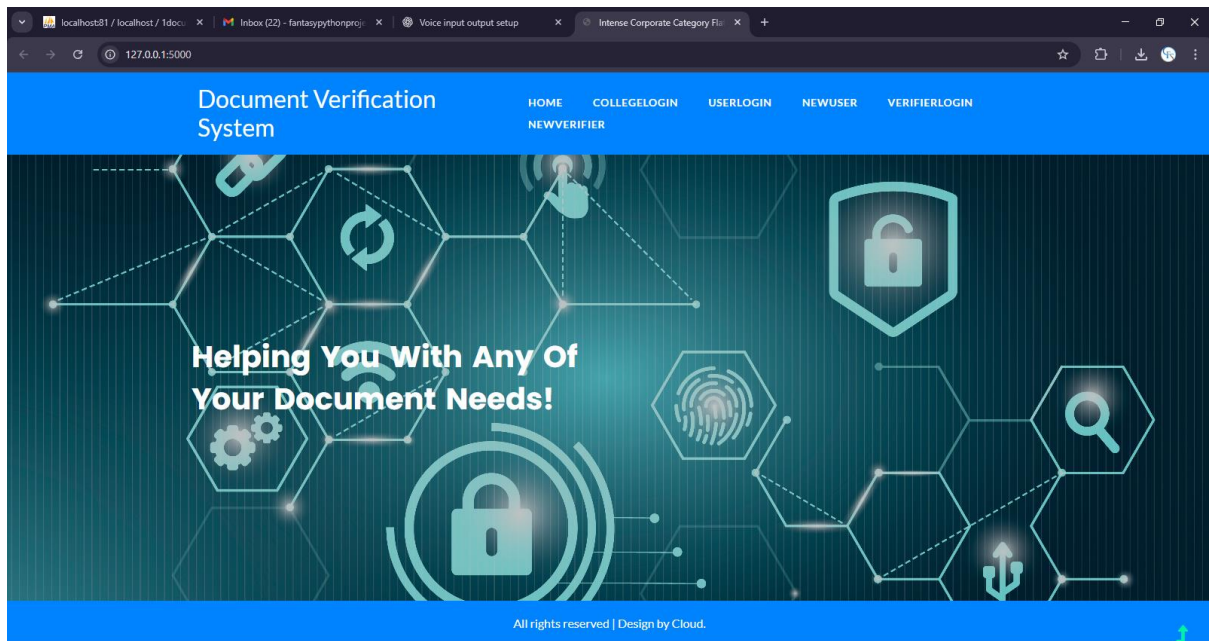


Fig. No. 10.1 Home Page

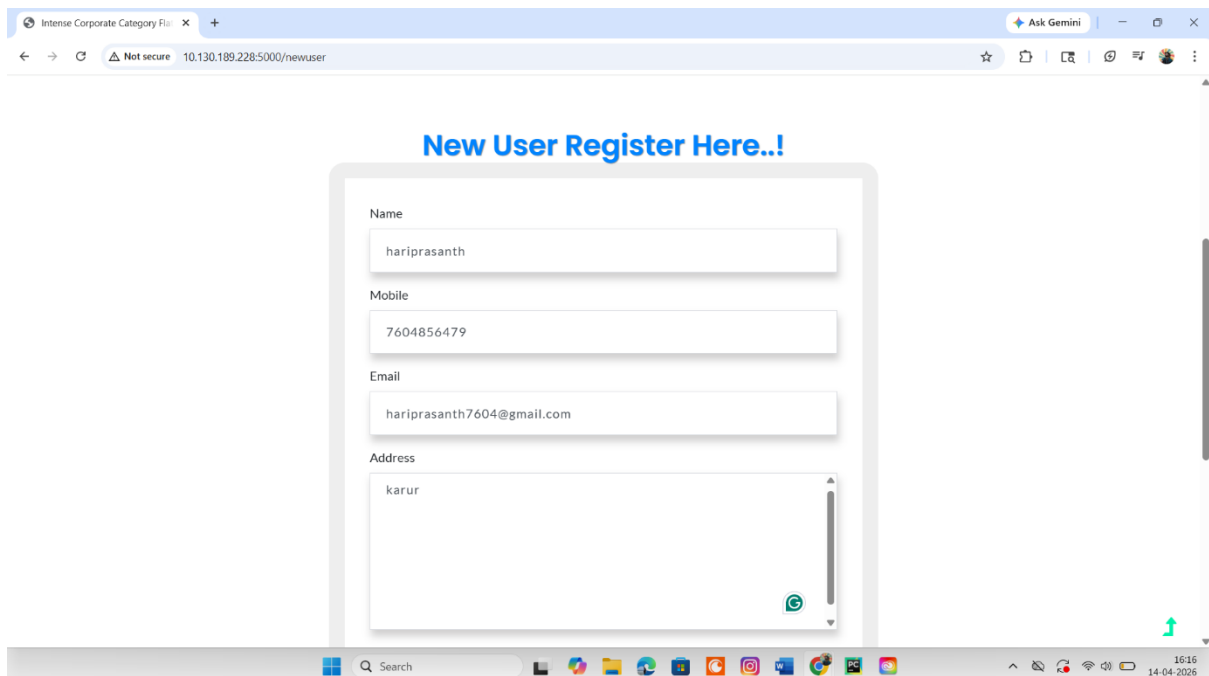


Fig. No. 10.2 New User Register

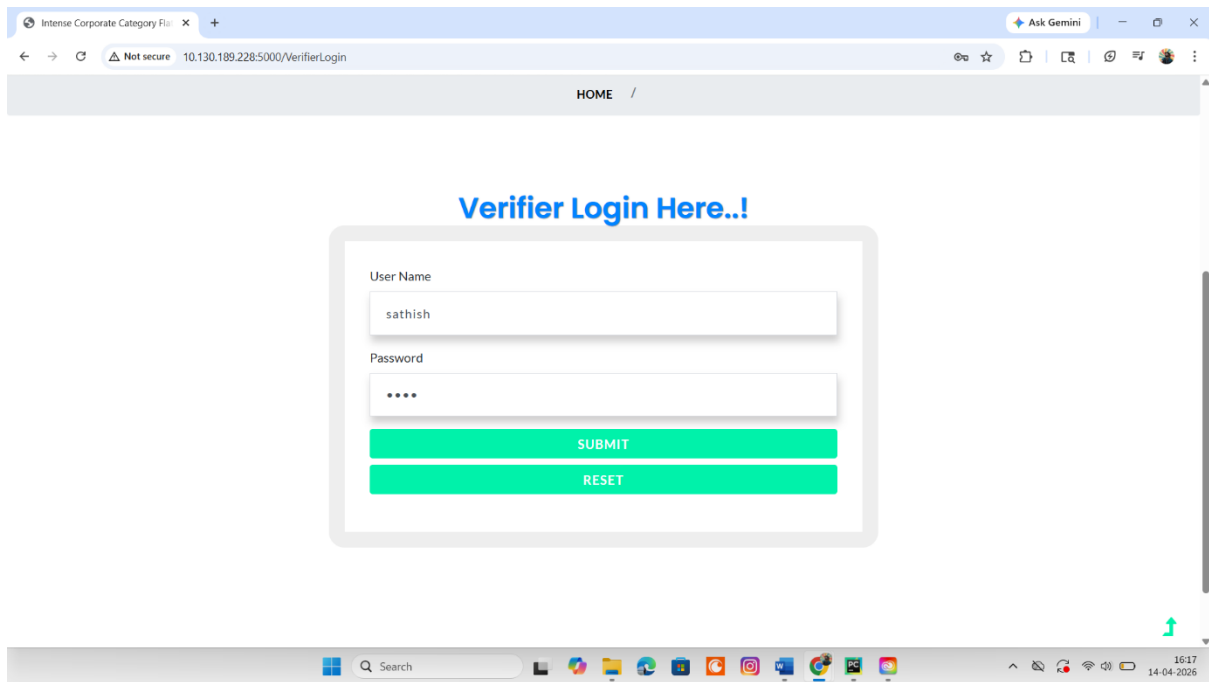


Fig. No. 10.3 Verifier Page

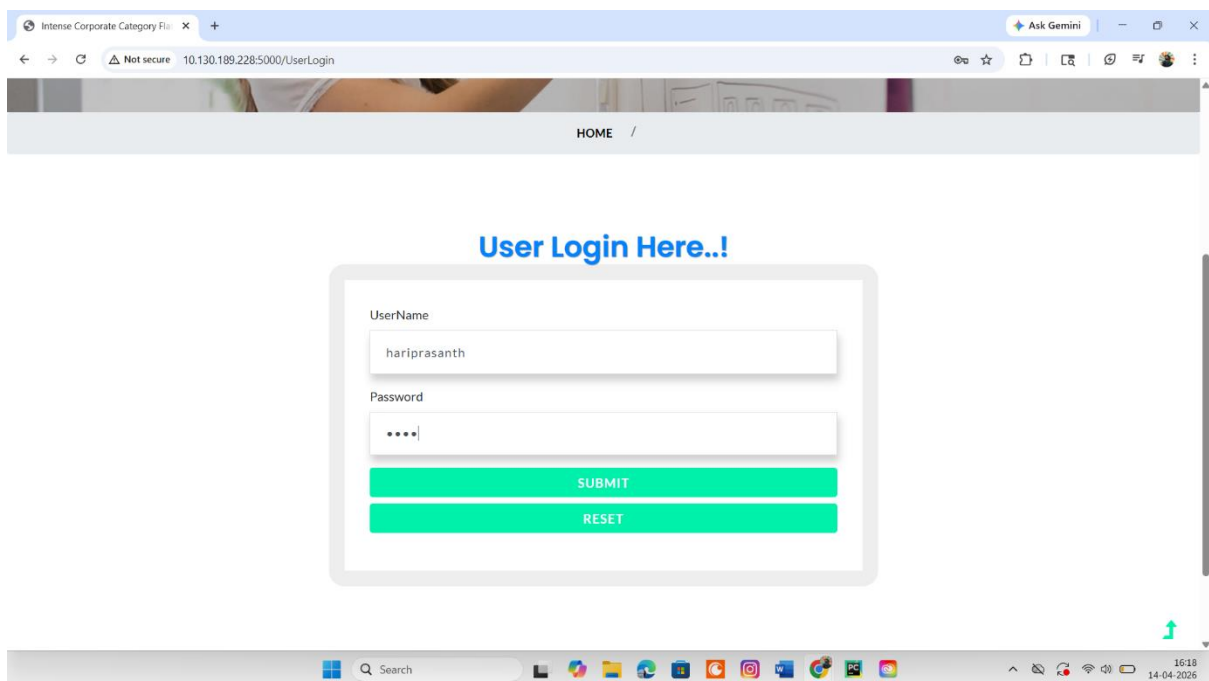


Fig. No. 10.4 User Login Page

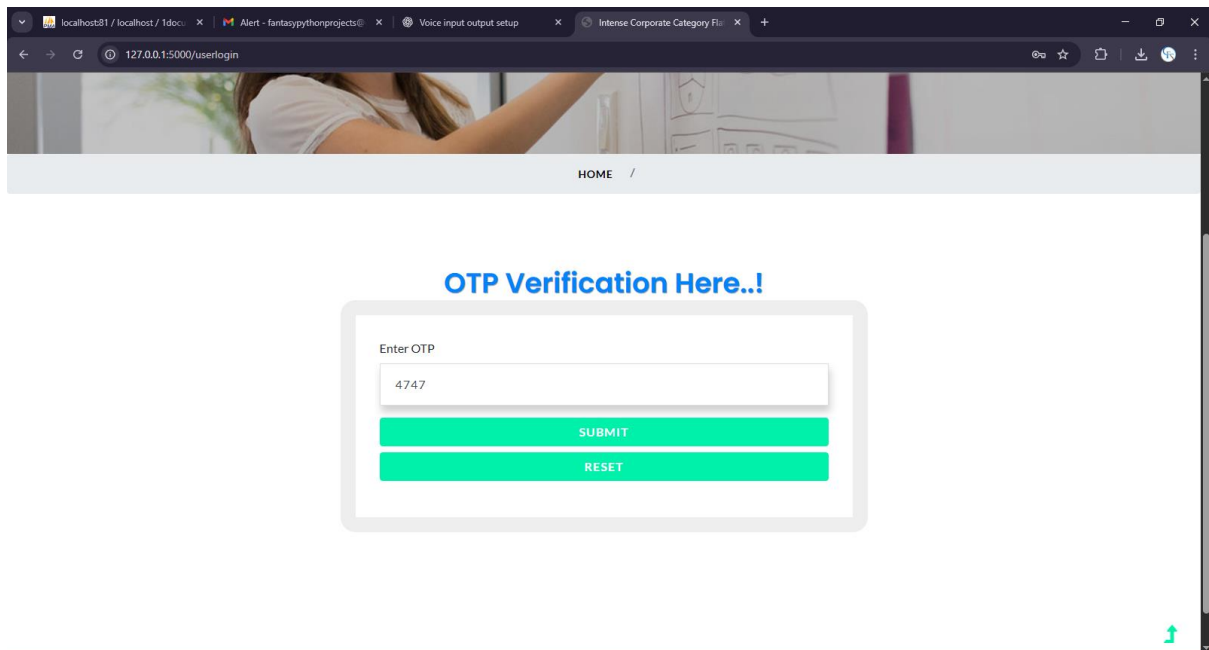


Fig. No. 10.5 OTP Verification Page

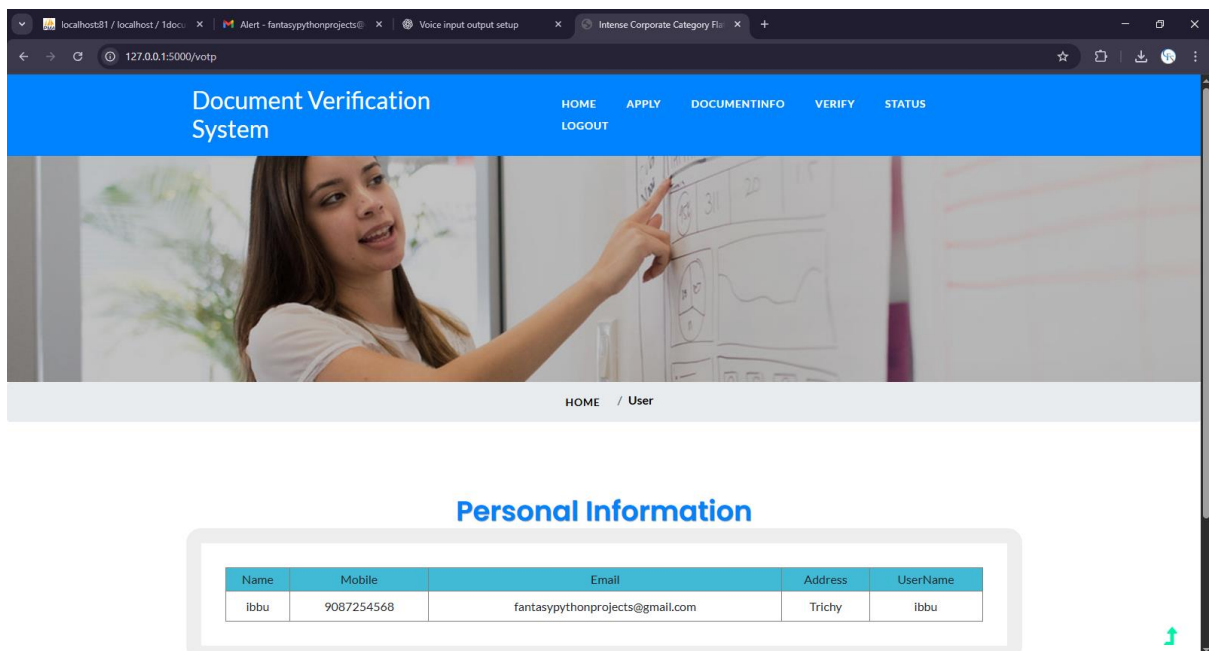


Fig. No. 10.6 Personal Information Page

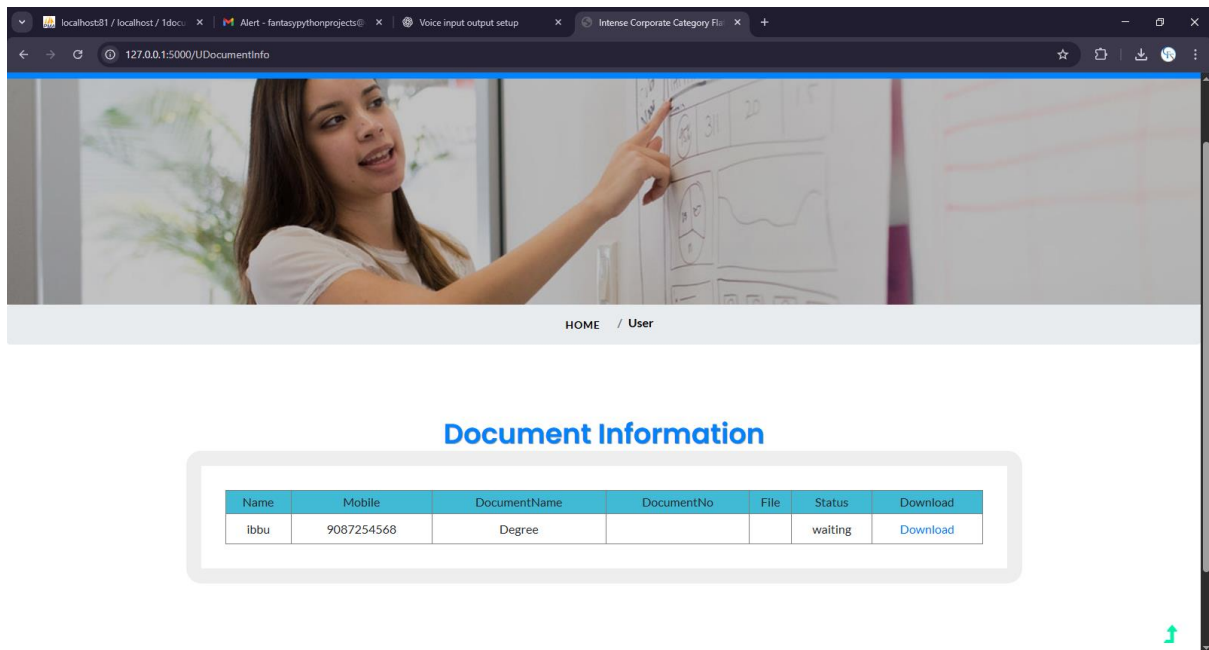


Fig. No. 10.7 Document Information Page

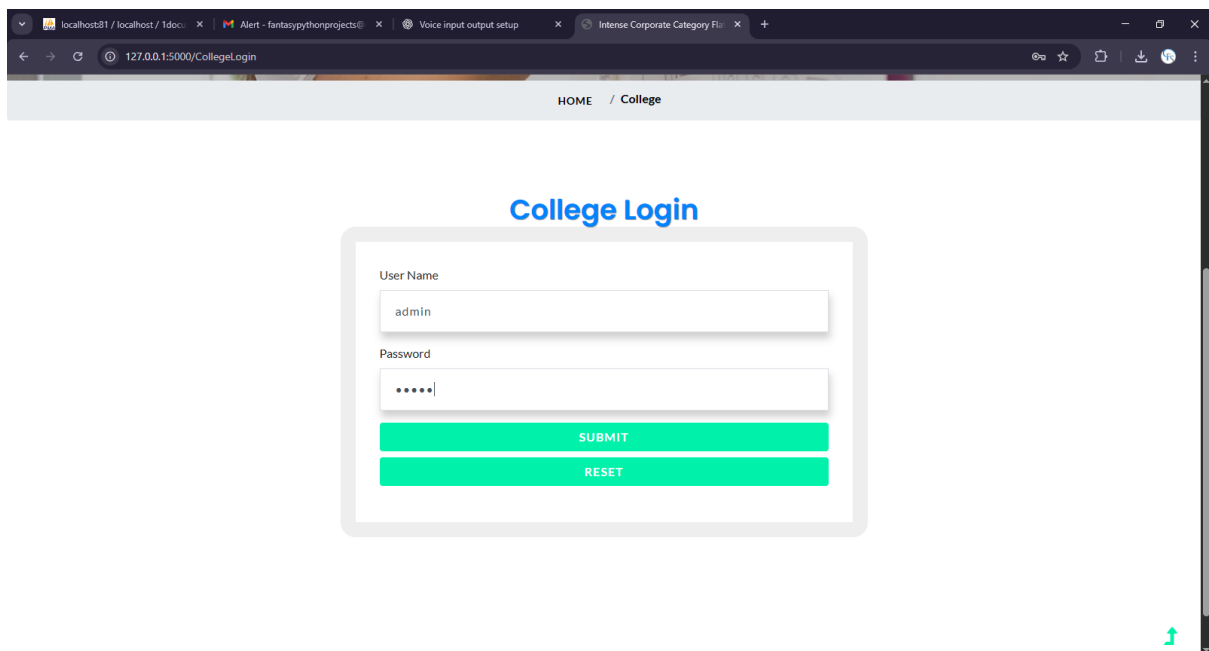


Fig. No. 10.8 College Login Page

CHAPTER 11

REFERENCES

1. Alshehri, Suhair, et al. "Dynamic secure access control and data sharing through trusted delegation and revocation in a blockchain-enabled cloud-IoT environment." *IEEE Internet of Things Journal* 10.5 (2022): 4239–4256.
2. Anisetti, Marco, et al. "Rethinking certification for trustworthy machine-learning-based applications." *IEEE Internet Computing* 27.6 (2023): 22–28.
3. Ashfaq, Tehreem, et al. "An efficient and secure energy trading approach with machine learning technique and consortium blockchain." *Sensors* 22.19 (2022): 7263.
4. Chanekar, Niraj, et al. "Fake Certificate Detection by using Blockchain." (2023).
5. Chen, Liang, et al. "Blockchain-based data integrity verification for large-scale IoT data." *IEEE Access* 7 (2019): 164996–165005.
6. Ghani, Rana F., et al. "Blockchain-based student certificate management and system sharing using Hyperledger Fabric platform." *Periodicals of Engineering and Natural Sciences* 10.2 (2022): 207–218.
7. Grech, Alexander, and Anthony F. Camilleri. "Blockchain in education." *Publications Office of the European Union* (2017).
8. Lutfiani, Ninda, et al. "Academic certificate fraud detection system framework using blockchain technology." *Blockchain Frontier Technology* 1.2 (2022): 55–64.
9. Matus, Kira J. M., and Michael Veale. "Certification systems for machine learning: Lessons from sustainability." *Regulation & Governance* 16.1 (2022): 177–196.

10. Rustemi, Avni, et al. "A systematic literature review on blockchain-based systems for academic certificate verification." *IEEE Access* 11 (2023): 64679–64696.
11. Said, Said Hamisi, et al. "A blockchain-based conceptual model to address educational certificate verification challenges in Tanzania." *Engineering, Technology & Applied Science Research* 13.5 (2023): 11691–11704.
12. Sharples, Mike, and John Domingue. "The blockchain and kudos: A distributed system for educational record, reputation and reward." *Computers & Education* 127 (2018): 162–174.
13. Turkanović, Muhamed, et al. "EduCTX: A blockchain-based higher education credit platform." *IEEE Access* 6 (2018): 5112–5127.
14. Zaman, Nihat, Işıl Karabey Aksakallı, and Nursena Baygın. "Digital Certificate Security: A Blockchain-based Approach for Fraud Prevention and Verification." *Bitlis Eren Üniversitesi Fen Bilimleri Dergisi* 12.4: 1128–1138.
15. Zheng, Zibin, et al. "Blockchain challenges and opportunities: A survey." *IEEE International Journal of Web and Grid Services* 14.4 (2018): 352–375.